

Cluster analysis: basic concepts and methods

Imagine that you are the director of Customer Relationships at a retail company. Managing millions of customers one by one is inefficient and ineffective. You would like to organize all customers of the company into a small number of groups so that each group can be assigned to a different manager. Strategically, you would like that the customers in each group are as similar as possible. Two customers having very different business patterns should not be placed in the same group. Your intention behind this business strategy is to develop customer relationship campaigns that specifically target each group, based on common features shared by the customers in the group. What kind of data mining techniques can help you accomplish this task?

Unlike in classification, the class label (i.e., the group-id in this context) of each customer is unknown in this new task. You need to *discover* these groupings. Given a large number of customers and many attributes describing customer profiles, it can be costly or even infeasible to manually study the data and come up with a way to partition the customers into strategic groups. You need a *clustering* tool to help.

Clustering is the process of grouping a set of data objects into multiple groups or *clusters* so that objects within a cluster have high similarity, but are dissimilar to objects in other clusters. Dissimilarities and similarities are assessed based on the attribute values describing the objects and often involve distance measures.¹ Clustering as a data mining tool has its roots in many application areas, such as biology, security, business intelligence, and Web search.

This chapter presents the basic concepts and methods of cluster analysis. In Section 8.1, we introduce the basic concept of clustering and study the requirements of clustering methods for massive amounts of data and various applications. You will learn several basic clustering techniques, organized into several categories, namely *partitioning methods* (Section 8.2), *hierarchical methods* (Section 8.3), and *density-based and grid-based methods* (Section 8.4). In Section 8.5, we discuss how to evaluate clustering methods. A discussion of advanced methods of clustering is reserved for Chapter 9.

8.1 Cluster analysis

This section sets up the groundwork for studying cluster analysis. Section 8.1.1 defines cluster analysis and presents examples where clustering is useful. In Section 8.1.2, you will learn aspects for comparing

¹ Data similarity and dissimilarity are discussed in detail in Chapter 2. You may want to refer to the corresponding section for a quick review.

clustering methods, as well as requirements for clustering. An overview of basic clustering techniques is presented in Section 8.1.3.

8.1.1 What is cluster analysis?

Cluster analysis or simply **clustering** is the process of partitioning a set of data objects (or observations) into subsets. Each subset is a **cluster**, such that objects in a cluster are similar to one another, yet dissimilar to objects in other clusters. The set of clusters resulting from a cluster analysis can be referred to as a **clustering**. In this context, different clustering methods may generate different clusterings on the same data set. The same clustering method equipped with different parameters or even different initializations may also produce different clusterings. Such partitioning is not performed by humans, but by a clustering algorithm. Hence, clustering is useful in that it can lead to the discovery of previously unknown groups within the data.

Cluster analysis has been widely used in many applications such as business intelligence, image pattern recognition, Web search, biology, and security. For example, in business intelligence, clustering can be used to organize a large number of customers into groups, where customers within a group share strong similar characteristics. This facilitates the development of business strategies for enhanced customer relationship management. Moreover, consider a consultant company with a large number of projects. To improve project management, such as project delivery and outcome quality control, clustering can be applied to partition projects into categories based on similarities in, for example, business scenarios, customers, expertise required, period and size, so that project auditing and diagnosis can be conducted effectively.

In image recognition, as another example, clustering can be used to discover clusters or “subclasses” in photos. One application is to automatically group photos according to faces recognized in the images so that the photos of the same person may likely come together into a group. Here, we do not have to specify and label the persons in the photos beforehand, and thus a classification method cannot be applied. A clustering method can use faces as features and partition photos into groups so that the faces in the same group are similar and the faces in different groups are dissimilar. Moreover, typically there are many different ways to organize photos. Clustering can help automatically identify significant features and suggest meaningful ways to organize photos into groups accordingly. For example, a group of scenic pictures may be formed using the features of blue sky and beach, whereas another group may share the theme of snow, and a third group highlights group photos with many faces.

Clustering has also found many applications in Web search. For example, a keyword search may often return a large number of hits (i.e., pages relevant to the search) due to the extremely large number of web pages. Clustering can be used to organize the search results into groups and present the results in a concise and easily accessible way. Moreover, clustering techniques have been developed to cluster documents into topics, which are commonly used in information retrieval practice.

As a data mining function, cluster analysis can be used as a standalone tool to gain insight into the distribution of data, to observe the characteristics of each cluster, and to focus on a particular set of clusters for further analysis. Alternatively, it may serve as a preprocessing step for other algorithms, such as characterization, attribute subset selection, and classification, which would then operate on the detected clusters and the selected attributes or features.

Because a cluster is a collection of data objects that are similar to one another within the cluster and dissimilar to objects in other clusters, a cluster of data objects can be treated as an implicit class.

In this sense, clustering is sometimes called **automatic classification** or **unsupervised classification**. Again, a critical difference here is that clustering can automatically find the groupings. This is a distinct advantage of cluster analysis.

Clustering is also called **data segmentation** in some applications because clustering partitions large data sets into groups according to their *similarity*. Clustering can also be used for **outlier detection**, where outliers (values that are “far away” from any cluster) may be more interesting than common cases. Applications of outlier detection include the detection of credit card frauds and the monitoring of criminal activities in electronic commerce. For example, exceptional cases in credit card transactions, such as very expensive and infrequent purchases at unusual locations, may be of interest as possible fraudulent activities. Outlier detection is the subject of Chapter 11.

Data clustering is under vigorous development. Contributing areas of research include data mining, statistics, machine learning and deep learning, spatial database technology, information retrieval, Web search, biology, marketing, and many other application areas. Owing to the huge amounts of data collected in databases, cluster analysis has become a highly active topic in data mining research.

As a branch of statistics, cluster analysis has been extensively studied, with the main focus on *distance-based cluster analysis*. Cluster analysis tools based on k -means, k -medoids, and several other methods also have been built into many statistical analysis software packages or systems, such as S-Plus, SPSS, and SAS. In machine learning, recall that classification is known as supervised learning because the class label information is given, that is, the learning algorithm is supervised in that it is told the class membership of each training tuple. Clustering is known as **unsupervised learning** because the class label information is not present. For this reason, clustering is a form of **learning by observation**, rather than *learning by examples*. In data mining, efforts have focused on finding methods for efficient and effective cluster analysis in *large data sets*. Active themes of research focus on the *scalability* of clustering methods, the effectiveness of methods for clustering *complex shapes* (e.g., nonconvex) and *types of data* (e.g., text, graphs, and images), *high-dimensional* clustering techniques (e.g., clustering objects with thousands or even millions of features), and methods for clustering *mixed numerical and nominal data* in large data sets.

8.1.2 Requirements for cluster analysis

Clustering is a challenging research field. In this section, you will learn about the requirements for clustering as a data mining tool, as well as aspects that can be used for comparing clustering methods.

When we think about employing a clustering method, what requirements should we consider for the method? The following are typical requirements of clustering in data mining.

- **Ability to deal with various kinds of data objects:** Many algorithms are designed to cluster numeric (interval-based) data objects. However, applications may require clustering objects based on a mixed data types, such as binary, nominal (categorical), ordinal, and numerical data, as well as data objects of various kinds, such as text, graphs, sequences, images, and videos.
- **Scalability:** Many clustering algorithms work well on small data sets containing fewer than several hundred data objects; however, a large database may contain millions or even billions of objects, such as in Web search scenarios. Clustering on only a sample of a given large data set may lead to biased results. Therefore highly scalable clustering algorithms are needed.
- **Discovery of clusters with arbitrary shape:** Many clustering algorithms determine clusters based on Euclidean or Manhattan distance measures (Chapter 2). Algorithms based on such distance mea-

tures tend to find spherical clusters with similar size and density. However, a cluster could be of any shape. For example, we may want to use clustering to find the frontier of a running forest fire in a satellite image, which is often not spherical. It is important to develop algorithms that can detect clusters of arbitrary shape.

- **Requirements for domain knowledge to determine input parameters:** Many clustering algorithms require users to provide domain knowledge in the form of input parameters such as the desired number of clusters. Consequently, the clustering results may be sensitive to such parameters. Parameters are often hard to determine, especially for data sets of high dimensionality where users have yet to grasp a deep understanding of their data. Requiring the specification of domain knowledge not only burdens users, but also makes the quality of clustering difficult to control. Clustering algorithms that do not heavily rely on domain knowledge input or can help users explore domain knowledge are highly preferable.
- **Ability to deal with noisy data:** Most real-world data sets contain outliers and/or missing, unknown, or erroneous data. For example, data collected by physical sensors is often noisy. Clustering algorithms can be sensitive to such noise and may produce poor-quality clusters. Therefore, we need clustering methods that are robust to noise.
- **Incremental clustering and insensitivity to input order:** In many applications, incremental updates (representing newer data) may arrive at any time. Some clustering algorithms cannot incorporate incremental updates into existing clustering structures and, instead, have to recompute a new clustering from scratch. Clustering algorithms may also be sensitive to the input data order. That is, given a set of data objects, clustering algorithms may return dramatically different clusterings depending on the order in which the objects are presented. Incremental clustering algorithms and algorithms that are insensitive to the input order are needed.
- **Capability of clustering high-dimensionality data:** A data set can contain numerous dimensions or attributes. When clustering documents, for example, each keyword can be regarded as a dimension, and there are often thousands of keywords. Most clustering algorithms are good at handling low-dimensional data such as data sets involving only two or three dimensions. Finding clusters of data objects in a high-dimensional space is challenging, especially considering that such data can be very sparse and highly skewed.
- **Constraint-based clustering:** Real-world applications may need to perform clustering under various kinds of constraints. Suppose that your task is to choose the locations for a given number of new electric vehicle charging stations in a city. To decide upon this, you may cluster potential charging needs while considering constraints such as available spaces, electricity networks, and the river and highway networks in a city. A challenging task is to find data groups with good clustering behaviors that satisfy specified constraints.
- **Interpretability and usability:** Users want clustering results to be interpretable, comprehensible, and usable. That is, clustering may need to be tied with specific semantic interpretations and applications. It is important to study how an application goal may influence the selection of clustering features and clustering methods.

Given one data set, using different clustering methods or using different parameters or initializations, we may be able to obtain different clusterings. How can we evaluate and compare the clusterings? The following are the orthogonal aspects with which clustering methods can be compared.

- **Single vs. multilevel clustering:** In many clustering methods, all the objects are partitioned so that no hierarchy exists among the clusters. That is, all the clusters are at the same level conceptually. Such a method is useful, for example, for partitioning customers into groups so that each group has its own manager. Alternatively, other methods partition data objects hierarchically, where clusters can be formed at different semantic levels. For example, in text mining, we may want to organize a corpus of documents into multiple general topics, such as “politics” and “sports,” each of which may have subtopics. For instance, “football,” “basketball,” “baseball,” and “hockey” can exist as subtopics of “sports.” The latter four subtopics are at a lower level in the hierarchy than “sports.”
- **Separation of clusters:** Some methods partition data objects into mutually exclusive clusters. In some other situations, the clusters may not be exclusive, that is, a data object may belong to more than one cluster. For example, when clustering documents into topics, a document may be related to multiple topics. Thus the topics as clusters may not be exclusive.
- **Similarity measure:** Some methods determine the similarity between two objects by the distance between them. Such a distance can be defined on a Euclidean space, a road network, a vector space, or some other space. For some applications, similarity may also be defined by other means such as connectivity based on density or contiguity and thus may or may not rely on the absolute distance between two objects. Similarity measures play a fundamental role in the design of clustering methods. While distance-based methods can often take advantage of some computation and optimization techniques, density- and continuity-based methods can often find clusters of arbitrary shape.
- **Clustering in full space vs. subspace:** Many clustering methods search for clusters within the entire given data space. These methods are useful for low-dimensionality data sets. With high-dimensional data, however, there can be many irrelevant attributes, which can make similarity measurements unreliable. Consequently, clusters found in the full space are often meaningless. It is often better to instead search for clusters within different subspaces of the same data set. *Subspace clustering* discovers both clusters and subspaces (often of low dimensionality) containing interesting clusters.

To conclude, clustering algorithms have a series of requirements. These factors include the ability to deal with different kinds of data objects, scalability, robustness to noisy data, incremental updates, clusters of arbitrary shape, and constraints. Interpretability and usability are also important. In addition, clustering methods can differ with respect to single vs. multilevel clustering, whether or not clusters are mutually exclusive, the similarity measures used, and whether or not subspace clustering is performed.

8.1.3 Overview of basic clustering methods

There are many clustering algorithms in the literature. It is difficult to provide a crisp categorization of clustering methods because these categories may overlap so that a method may have features from several categories. Nevertheless, it is useful to present a relatively organized picture of clustering methods. In general, the major fundamental clustering methods can be classified into the following categories, which are discussed in the rest of this chapter.

Partitioning methods: Given a set of n objects, a partitioning method constructs k ($k \leq n$) partitions of the data, where each partition represents a cluster. That is, it divides the data into k groups such that each group must contain at least one object. Typically, k is set to a small number, that is, $k \ll n$. In other words, a partitioning method conducts one-level partitioning on data sets. The basic partitioning methods typically adopt *exclusive cluster separation*. That is, each object must

belong to exactly one group. This requirement may be relaxed. For example, in fuzzy partitioning techniques an object may take probabilities to belong to more than one cluster. References to such techniques are given in the bibliographic notes (Section 8.8).

Most partitioning methods are distance-based. Given k , the number of partitions to construct, a partitioning method creates an initial partitioning. It then uses an **iterative relocation technique** that attempts to improve the partitioning by moving objects from one group to another. The general criterion of a good partitioning is that objects in the same cluster are “close” or related to each other, whereas objects in different clusters are “far apart” or very different. There are various kinds of other criteria for judging the quality of partitions. Traditional partitioning methods can be extended for subspace clustering rather than searching the full data space. This is useful when there are many attributes and the data is sparse.

Achieving global optimality in partitioning-based clustering is often computationally prohibitive, potentially requiring an exhaustive enumeration of all the possible partitions. Instead, most applications adopt popular heuristic methods, such as greedy approaches like the *k-means* and the *k-medoids* algorithms, which progressively improve the clustering quality and approach a local optimum. These heuristic clustering methods work well for finding spherical-shaped clusters in small- to medium-sized data sets. To find clusters with complex shapes and for very large data sets, partitioning-based methods need to be extended. Partitioning-based clustering methods are studied in depth in Section 8.2.

Hierarchical methods: A hierarchical method creates a hierarchical decomposition of a given set of data objects. A hierarchical method can be classified as being either *agglomerative* or *divisive*, based on how the hierarchical decomposition is formed. The *agglomerative approach*, also called the *bottom-up* approach, starts with each object forming a separate group. It successively merges the objects or groups close to one another, until all the groups are merged into one (the topmost level of the hierarchy), or a termination condition holds. The *divisive approach*, also called the *top-down* approach, starts with all the objects in the same cluster. In each successive iteration, a cluster is split into smaller clusters, until eventually each object is in one cluster, or a termination condition holds.

Hierarchical clustering methods can be distance-, density-, or continuity-based. Various extensions of hierarchical methods consider clustering in subspaces as well.

Hierarchical methods suffer from the fact that once a step (merge or split) is done, it can never be reverted. This rigidity is useful in that it leads to smaller computation costs by not having to worry about a combinatorial number of different choices. Such techniques cannot correct erroneous decisions; however, methods for improving the quality of hierarchical clustering have been proposed.

Hierarchical clustering methods are studied in Section 8.3.

Density-based and grid-based methods: Most partitioning methods cluster objects based on the distance between objects. Such methods can find only spherical-shaped clusters and encounter difficulty in discovering clusters of arbitrary shapes. Other clustering methods have been developed based on the notion of *density*. Their general idea is to continue growing a given cluster as long as the density (number of objects or data points) in the “neighborhood” exceeds some threshold. For example, for each data point within a given cluster, the neighborhood of a given radius has to contain at least a minimum number of points. Such a method can be used to filter out noise or outliers and discover clusters of arbitrary shape.

Density-based methods can divide a set of objects into multiple exclusive clusters, or a hierarchy of clusters. Typically, density-based methods consider exclusive clusters only, and do not consider fuzzy clusters. Moreover, density-based methods can be extended from full space to subspace clustering.

One way to implement the idea of density-based clustering is grid-based methods, which quantize the object space into a finite number of cells that form a grid structure. All the clustering operations are performed on the grid structure (i.e., on the quantized space). For example, the dense cells, that is, those cells each containing a sufficient number of data points, are considered components of clusters, and are used to assemble clusters. The main advantage of the grid-based methods is the fast processing time, which is typically independent of the number of data objects and dependent only on the number of cells in each dimension in the quantized space. Using grids is often an efficient approach to many spatial data mining problems, including clustering. In addition to density-based clustering, grid-based methods can be integrated with other clustering methods, such as hierarchical methods. Density-based and grid-based clustering methods are studied in Section 8.4.

Some clustering algorithms integrate the ideas of several clustering methods, so that it is sometimes difficult to classify a given algorithm as uniquely belonging to only one clustering method category. Furthermore, some applications may have clustering criteria that require an integration of several clustering techniques.

In the following sections, we examine representative clustering methods in detail. Advanced clustering methods and related issues are discussed in Chapter 9.

8.2 Partitioning methods

The simplest and most fundamental version of cluster analysis is partitioning. In partitioning clustering, we organize the objects in a given set into several exclusive groups or clusters. Each cluster can be typified by a representative. In other words, each object o can be assigned to the cluster whose representative that o is the closest or most similar to. To keep the problem specification concise, we can assume that the number of expected clusters is given. This parameter is the starting point for partitioning methods. There are two foremost technical issues in partitioning methods. First, how can we decide the representatives of clusters? Second, how can we measure the distance or similarity between objects or between objects and representatives.

Formally, given a data set, D , of n objects, and k , the number of clusters to form, a **partitioning algorithm** organizes the objects into k partitions ($k \leq n$), where each partition represents a cluster. The clusters are formed to optimize an objective partitioning criterion, such as a dissimilarity function based on distance, so that the objects within a cluster are “similar” to one another and “dissimilar” to the objects in other clusters in terms of the data set attributes.

In this section you will learn the partitioning methods. We will start with the most prominent partitioning method, k -means (Section 8.2.1). Then, in Section 8.2.2 we will look at a series of variations of partitioning methods to address different types of data and different application scenarios. Last, we will discuss kernel k -means, an advanced version of partitioning method that can explore nonlinear separability between clusters in high dimensional data.

8.2.1 *k*-Means: a centroid-based technique

Suppose a data set, D , contains n objects in Euclidean space. Partitioning methods distribute the objects in D into k clusters, $C_1, \dots, C_k$, that is, $C_i \subset D$, $|C_i| \geq 1$, and $C_i \cap C_j = \emptyset$ for $(1 \leq i, j \leq k, i \neq j)$. Each cluster is required to have at least one object. An objective function is used to assess the partitioning quality so that objects within a cluster are similar to one another but dissimilar to objects in other clusters. This is, the objective function aims for high intracluster similarity and low intercluster similarity.

A centroid-based partitioning technique uses the *centroid* of a cluster, C_i , as the representative of that cluster. Conceptually, the centroid of a cluster is its center point. The centroid can be defined in various ways such as by the mean or medoid of the objects (or points) assigned to the cluster. The difference between an object $p \in C_i$ and c_i , the representative of the cluster, is measured by $\text{dist}(p, c_i)$, where $\text{dist}(x, y)$ is the Euclidean distance between two points x and y . The quality of cluster C_i can be measured by the **within-cluster variation**, which is the sum of *squared error* between all objects in C_i and the centroid c_i , defined as

$$E = \sum_{i=1}^k \sum_{p \in C_i} \text{dist}(p, c_i)^2, \quad (8.1)$$

where E is the sum of the squared error for all objects in the data set; p is the point in space representing a given object; and c_i is the centroid of cluster C_i (both p and c_i are multidimensional). In other words, for each object in each cluster, the distance from the object to its cluster center is squared, and the distances are summed. This objective function tries to make the resulting k clusters as compact and separate as possible. The task of partitioning clustering can be modeled as to minimize the within-cluster variation (Eq. (8.1)) among all possible assignments of objects into clusters.

Minimizing the within-cluster variation is computationally challenging. In the worst case, we would have to enumerate the number of all possible partitionings. It is easy to see that the number of all possible partitionings is exponential to the number of objects. (This is left to be an exercise.) It has been shown that the problem is NP-hard in the general Euclidean space even for two clusters (i.e., $k = 2$). To overcome the prohibitive computational cost for the exact solution, greedy approaches are often used in practice. A prime example is the *k*-means algorithm, which is simple and commonly used.

“How does the *k*-means algorithm work?” The *k*-means algorithm defines the centroid of a cluster as the mean value of the points within the cluster. It proceeds as follows. First, it randomly selects k objects from D , each of which initially represents a cluster mean or center. For each of the remaining objects, an object is assigned to the cluster to which it is the most similar, based on the Euclidean distance between the object and the chosen means. The *k*-means algorithm then iteratively improves the within-cluster variation. For each cluster, it computes the new mean using the objects assigned to the cluster in the previous iteration. All the objects are then reassigned using the updated means as the new cluster centers. The iterations continue until the assignment is stable, that is, the clusters formed in the current round are the same as those formed in the previous round. The *k*-means procedure is summarized in Fig. 8.1.

Example 8.1. Clustering by *k*-means partitioning. Consider a set of objects located in 2-D space, as depicted in Fig. 8.2(a). Let $k = 3$, that is, the user would like to partition the objects into three clusters.

According to the algorithm in Fig. 8.1, we arbitrarily choose three objects as the three initial cluster centers, where cluster centers are marked by a +. Each object is assigned to a cluster based on the cluster

Algorithm: k -means. The k -means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.

Input:

- k : the number of clusters,
- D : a data set containing n objects.

Output: A set of k clusters.

Method:

- (1) arbitrarily choose k objects from D as the initial cluster centers;
- (2) **repeat**
- (3) (re)assign each object to the cluster to which the object is the most similar;
- (4) update the cluster centers, that is, calculate the mean value of the objects for each cluster;
- (5) **until** no change;

FIGURE 8.1

The k -means partitioning algorithm.

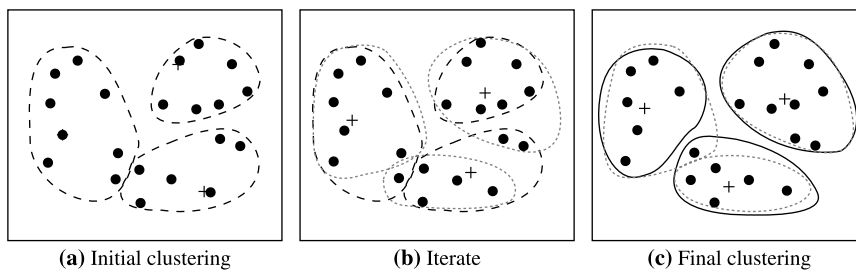


FIGURE 8.2

Clustering of a set of objects using the k -means method; for (b) update cluster centers and reassign objects accordingly (the mean of each cluster is marked by a +).

center to which it is the nearest. Such an assignment forms silhouettes encircled by dotted curves, as shown in Fig. 8.2(a).

Next, the cluster centers are updated. That is, the mean value of each cluster is recalculated based on the current objects in the cluster. Using the new cluster centers, the objects are reassigned to the clusters based on which cluster center is the nearest. Such a reassignment forms new silhouettes encircled by dashed curves, as shown in Fig. 8.2(b).

This process iterates, leading to Fig. 8.2(c). The process of iteratively reassigning objects to clusters to improve the partitioning is referred to as *iterative relocation*. Eventually, no reassignment of the objects in any cluster occurs and so the process terminates. The resulting clusters are returned by the clustering process. $\square$

The k -means method is not guaranteed to converge to the global optimum and often terminates at a local optimum. The results may depend on the initial random selection of cluster centers. (You will be asked to give an example to show this as an exercise.) To obtain good results in practice, it is common

to run the k -means algorithm multiple times with different initial cluster centers. The clustering with the smallest within-cluster variation should be returned as the final result.

The time complexity of the k -means algorithm is $O(nkt)$, where n is the total number of objects, k is the number of clusters, and t is the number of iterations. Normally, $k \ll n$ and $t \ll n$. Therefore the method is relatively scalable and efficient in processing large data sets.

As the simplest version of partitioning methods, the k -means method has several advantages. First, the k -means method is conceptually intuitive and relatively simple to implement. Indeed, the k -means method is included in many software toolkits and open source suites in statistics, data mining, and machine learning. Second, as analyzed, the k -means method is scalable to large data sets. The runtime is linear with respect to the data set size (i.e., the number of data objects), the number of clusters, and the number of iterations. Third, the k -means method is guaranteed to converge to some local optimum, and thus likely does not produce very poor results. Fourth, if a user has some domain knowledge about the possible locations of the clusters, the user can set the initial means and then run the iteration steps. In other words, the k -means method can take a warm-start. Last, the k -means method can take new observed data easily. That is, if some new data objects arrive after a certain number of iterations, the k -means method still can easily take those data into the next iteration, and the updated clustering can adapt to the new data.

The k -means method also has some limitations. First, a user has to manually specify the number of clusters. When a user is not familiar with a data set, it is not easy to set this parameter properly. Second, the effect of result clustering heavily depends on the choice of initial means. When the number of clusters is small, to overcome this limitation, one may run the k -means method multiple times with different initial means. However, when the number of clusters is large, even running the k -means method multiple times may not help to mitigate the issue, since it is unlikely all clusters produced in a run are all good. Third, as to be illustrated later (see Fig. 8.16 as an example), the k -means method may meet difficulty in finding clusters of substantially different sizes and density. Moreover, outliers may distract the centers of clusters (see Example 8.2 for demonstration). Last, since the Euclidean distance is used in the k -means method, when the dimensionality increases, the distance measure is mainly dominated by noise. In expectation, the distance between any two data objects in a high dimensional space is the same. Thus the k -means method cannot scale up to high dimensional data straightforwardly.

In the rest of the section, we will discuss some variations of the k -means method to address some of the above limitations. Some other limitations are unfortunately shared by the partitioning methods, and thus have to be tackled by introducing other types of clustering methods.

8.2.2 Variations of k -means

In order to tackle various limitations, there are multiple variants of the k -means method. In this subsection, we will study some of them that are popularly used in various applications.

k-Medoids: a representative object-based technique

The k -means algorithm is sensitive to outliers because such objects are far away from the majority of the data, and thus, when assigned to a cluster, they can dramatically distort the mean value of the cluster. This inadvertently affects the assignment of other objects to clusters. This effect is particularly exacerbated due to the use of the *squared-error* function (Eq. (8.1)), as observed in Example 8.2.

Example 8.2. A drawback of k -means. Consider six points in a 1-D space having values 1, 2, 3, 8, 9, 10, and 25, respectively. Intuitively, by visual inspection we may imagine the points partitioned into the clusters $\{1, 2, 3\}$ and $\{8, 9, 10\}$, where point 25 is excluded because it appears to be an outlier. How would k -means partition the values? If we apply k -means using $k = 2$ and Eq. (8.1), the partitioning $\{\{1, 2, 3\}, \{8, 9, 10, 25\}\}$ has the within-cluster variation

$$(1 - 2)^2 + (2 - 2)^2 + (3 - 2)^2 + (8 - 13)^2 + (9 - 13)^2 + (10 - 13)^2 + (25 - 13)^2 = 196,$$

given that the mean of cluster $\{1, 2, 3\}$ is 2 and the mean of $\{8, 9, 10, 25\}$ is 13. Compare this to the partitioning $\{\{1, 2, 3, 8\}, \{9, 10, 25\}\}$, for which k -means computes the within-cluster variation as

$$(1 - 3.5)^2 + (2 - 3.5)^2 + (3 - 3.5)^2 + (8 - 3.5)^2 + (9 - 14.67)^2 \\ + (10 - 14.67)^2 + (25 - 14.67)^2 = 189.67,$$

given that 3.5 is the mean of cluster $\{1, 2, 3, 8\}$ and 14.67 is the mean of cluster $\{9, 10, 25\}$. The latter partitioning has the lower within-cluster variation; therefore the k -means method assigns the value 8 to a cluster different from that containing values 9 and 10 due to the outlier point 25. Moreover, the center of the second cluster, 14.67, is substantially far from all the members in the cluster. $\square$

“How can we modify the k -means algorithm to diminish such sensitivity to outliers?” Instead of taking the mean value of the objects in a cluster as a reference point, we can pick actual objects to represent the clusters, using one representative object per cluster. Each remaining object is assigned to the cluster of which the representative object is the most similar. The partitioning method is then performed based on the principle of minimizing the sum of the dissimilarities between each object p and its corresponding representative object. That is, an **absolute-error criterion** is used, defined as

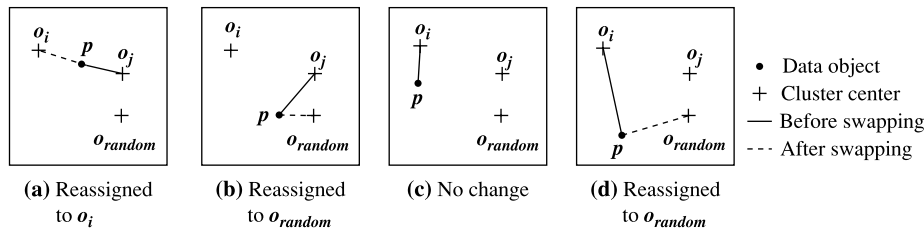
$$E = \sum_{i=1}^k \sum_{p \in C_i} \text{dist}(p, o_i), \quad (8.2)$$

where E is the sum of the absolute error for all objects p in the data set, and o_i is the representative object of C_i . This is the basis for the **k -medoids method**, which groups n objects into k clusters by minimizing the absolute error (Eq. (8.2)).

When $k = 1$, we can find the exact median in $O(n^2)$ time. However, when k is a general positive number, the k -medoid problem is NP-hard.

The **Partitioning Around Medoids (PAM)** algorithm (see Fig. 8.4 later) is a popular realization of k -medoids clustering. It tackles the problem in an iterative, greedy way. Like the k -means algorithm, the initial representative objects (called seeds) are chosen arbitrarily. We consider whether replacing a representative object by a nonrepresentative object would improve the clustering quality. All the possible replacements are tried out. The iterative process of replacing representative objects by other objects continues until the quality of the resulting clustering cannot be improved by any replacement. This quality is measured by a cost function of the average dissimilarity between an object and the representative object of its cluster.

Specifically, let $o_1, \dots, o_k$ be the current set of representative objects (i.e., medoids). To determine whether a nonrepresentative object, denoted by o_{random} , is a good replacement for a current medoid

**FIGURE 8.3**

Four cases of the cost function for k -medoids clustering.

o_j ($1 \leq j \leq k$), we calculate the distance from every object p to the closest object in the set $\{o_1, \dots, o_{j-1}, o_{\text{random}}, o_{j+1}, \dots, o_k\}$, and use the distance to update the cost function. The reassignments of objects to $\{o_1, \dots, o_{j-1}, o_{\text{random}}, o_{j+1}, \dots, o_k\}$ are simple. Suppose object p is currently assigned to a cluster represented by medoid o_j (Fig. 8.3a or b). Do we need to reassign p to a different cluster if o_j is being replaced by o_{random} ? Object p needs to be reassigned to either o_{random} or some other cluster represented by o_i ($i \neq j$), whichever is the closest. For example, in Fig. 8.3(a), p is closest to o_i and therefore is reassigned to o_i . In Fig. 8.3(b), however, p is closest to o_{random} and so is reassigned to o_{random} . What if, instead, p is currently assigned to a cluster represented by some other object o_i , $i \neq j$? Object p remains assigned to the cluster represented by o_i as long as p is still closer to o_i than to o_{random} (Fig. 8.3c). Otherwise, p is reassigned to o_{random} (Fig. 8.3d).

Each time a reassignment occurs, a difference in absolute error, E , is contributed to the cost function. Therefore the cost function calculates the *difference* in absolute-error value if a current representative object is replaced by a nonrepresentative object. The total cost of swapping is the sum of costs incurred by all nonrepresentative objects. If the total cost is negative, then o_j is replaced or swapped with o_{random} because the actual absolute-error E is reduced. If the total cost is positive, the current representative object, o_j , is considered acceptable, and nothing is changed in the iteration.

“Which method is more robust— k -means or k -medoids?” The k -medoids method is more robust than k -means in the presence of noise and outliers because a medoid is less influenced by outliers or other extreme values than a mean. However, the complexity of each iteration in the k -medoids algorithm is $O(k(n - k))$. For large values of n and k , such computation becomes very costly and much more costly than the k -means method. Both methods require the user to specify k , the number of clusters.

***k*-Modes: clustering nominal data**

One limitation of the k -means method is that it can be applied only when the mean of a set of objects is defined. This may not be the case in some applications such as when data with nominal attributes is involved. The ***k*-modes method** is a variant of k -means, which extends the k -means paradigm to cluster nominal data by replacing the means of clusters with modes.

Recall that the mode for a set of data is the value that occurs most frequently in the set. In order to use modes in clustering, we need a new way to compute the distance between two objects. Given two

Algorithm: k -medoids. PAM, a k -medoids algorithm for partitioning based on medoid or central objects.

Input:

- k : the number of clusters,
- D : a data set containing n objects.

Output: A set of k clusters.

Method:

- (1) arbitrarily choose k objects in D as the initial representative objects or seeds;
- (2) **repeat**
- (3) assign each remaining object to the cluster with the nearest representative object;
- (4) randomly select a nonrepresentative object, o_{random} ;
- (5) compute the total cost, S , of swapping representative object, o_j , with o_{random} ;
- (6) **if** $S < 0$ **then** swap o_j with o_{random} to form the new set of k representative objects;
- (7) **until** no change;

FIGURE 8.4

PAM, a k -medoids partitioning algorithm.

objects $\mathbf{x} = (x_1, \dots, x_l)$ and $\mathbf{y} = (y_1, \dots, y_l)$, we define the distance

$$dist(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^l d(x_i, y_i), \quad (8.3)$$

where $d(x, y) = 1$ if $x \neq y$ and otherwise 0. With this change, the sum of squared error (Eq. (8.1)) remains valid, where $\mathbf{c}_i$ is the representative of cluster i .

The k -modes method works largely the same way as the k -means method. First, it selects k initial modes, one for each cluster. Second, it allocates an object to the cluster whose mode is the closest to the object using the distance function in Eq. (8.3). Third, it updates the mode of each cluster. For a cluster i and dimension j , the mode is updated to the most frequent value on the dimension of all objects assigned to this cluster. If there are more than one such a value, that is, if two or more values of the same frequency happen most frequently in the cluster, we can randomly choose one. The k -modes method iterates the object allocation and mode update steps until either the sum of squared error (Eq. (8.1)) stabilizes or a given number of iterations are conducted.

The k -means and the k -modes methods can be integrated to cluster data with mixed numeric and nominal values. This is known as the k -prototype method. On each dimension, according to whether it is a numeric attribute or a nominal attribute, we can use either the absolute error $dist(x - y)$ or the mode difference $d(x, y) = 1$ if $x \neq y$ and otherwise 0. Since numeric attributes may have a much larger range in absolute error than the mode difference on nominal attributes, we can associate with each dimension a weight balancing the effect of each dimension. You will have an opportunity to explore the details of the k -prototype method in the exercise.

Initialization in partitioning methods

It is interesting that by choosing the initial cluster centers carefully, we may be able to not only speed up the convergence of the k -means algorithm, but also guarantee the quality of the final clustering results. For example, the k -means++ algorithm chooses the initial centers in the following steps. First, it chooses one center uniformly at random from the objects in the data set. Iteratively, for each object

p other than the chosen centers, it chooses the object as the new center at random with probability proportional to $D(p)^2$, where $D(p)$ is the distance from p to the closest center that has already been chosen. The iteration continues until k centers are chosen.

Extensive experimental results have shown that the k -means++ algorithm can speed up the clustering process by a factor from 2 in most cases. Moreover, the k -means++ algorithm guarantees an approximation ratio of $O(\log k)$; that is, the within-cluster variation obtained by k -means++ is not more than $O(\log k)$ times larger than the global optimum.

Estimating the number of clusters

The necessity for users to specify k , the number of clusters, in advance can be seen as a disadvantage. The desired number of k is often dependent on the shape and scale of the distribution of points in a data set and the desired clustering resolution of the user. There have been studies on how to estimate a desired number of clusters. For example, given a data set of n objects, let $B(k)$ and $W(k)$ be the sum of squares of the distances between and within clusters, respectively, when there are k clusters. The *Calinski-Harabasz index* is defined by

$$CH(k) = \frac{\frac{B(k)}{k-1}}{\frac{W(k)}{n-k}}. \quad (8.4)$$

The number of clusters k can be estimated by maximizing the Calinski-Harabasz index.

Gap statistic is another method to estimate the number of clusters. The sum of the pairwise distances for all points in a cluster C_i is

$$SD_{C_i} = \sum_{p, q \in C_i} dist(p, q).$$

If the data set is divided into k clusters, define

$$W_k = \sum_{i=1}^k \frac{SD_{C_i}}{2|C_i|},$$

which is the pooled within-cluster sum of squares around the cluster means. The *gap statistic* is

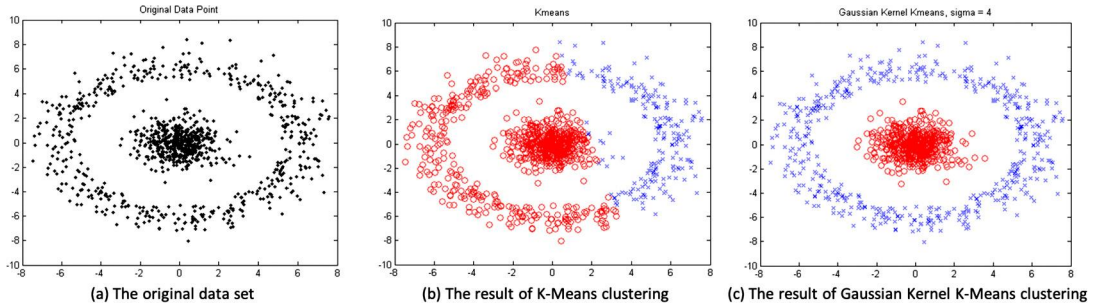
$$Gap_n(k) = E_n^*\{\log(W_k)\} - \log(W_k), \quad (8.5)$$

where E_n^* is the expectation under a sample of size n from the reference distribution, that is, the distribution producing the data set to be clustered. We can choose the value k that maximizes the gap statistic as the estimation of number of clusters.

In Section 8.5.2, we will introduce additional methods to estimate the number of clusters.

Applying feature transformation

The k -means method employing the Euclidean distance or any metric measures in general can only output convex clusters. Here, a cluster is convex if for any two points a and b belonging to the cluster, every point between the two points on the line connecting a and b also belongs to the cluster. Moreover, the k -means method employing any metric measure can only detect clusters that are linearly separable. That is, two clusters can be separated by a linear hyperplane.

**FIGURE 8.5**

Concave and not linearly separable clusters can be detected by kernel k -means.

In many applications, clusters may not be convex or linearly separable. In Fig. 8.5(a), one can easily see that there are two clusters, the points at the center form a cluster, which is convex. The other points form another cluster in a “ring” shape. If we apply k -means on the data set, specify the number of clusters $k = 2$ and use the Euclidean distance, the output is in Fig. 8.5(b). As you can see, k -means cuts the data set into two parts using a line. However, those two parts do not match the visual intuition.

Can we still use the k -means method to find clusters that are concave and not linearly separable? Indeed, kernel k -means is such a method. The general idea of kernel k -means is to map the data points in the original input space to a feature space of higher dimensionality where the points belonging to the same cluster are close to each other in the feature space. Explicitly defining a space of high dimensionality and mapping the points into that space is subtle and may be costly. Instead, a convenient way is to apply a kernel function to measure the distance between points.

Recall that Section 7.3.2 introduces the concept of kernel functions. For example, using the Gaussian radial basis function (RBF) kernel, we can calculate the distance between two points $\mathbf{x}$ and $\mathbf{y}$ by

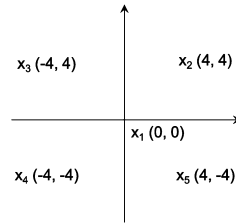
$$K(\mathbf{x}, \mathbf{y}) = e^{-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2}},$$

where $\|\mathbf{x} - \mathbf{y}\|^2$ is indeed the squared Euclidean distance between the two points, and σ is a free parameter. Clearly, the RBF kernel has the range between 0 and 1 and decreases with respect to the Euclidean distance.

How does a kernel function, such as the RBF kernel, transform the similarity among data points? Consider the five points in Fig. 8.6. The Euclidean distance matrix is

$$\begin{bmatrix} 0 & 5.66 & 5.66 & 5.66 & 5.66 \\ 5.66 & 0 & 8 & 11.31 & 8 \\ 5.66 & 8 & 0 & 8 & 11.31 \\ 5.66 & 11.31 & 8 & 0 & 8 \\ 5.66 & 8 & 11.31 & 8 & 0 \end{bmatrix}$$

where the value at the i th row and the j th column is the Euclidean distance between $\mathbf{x}_i$ and $\mathbf{x}_j$. Let $\sigma = 4$. We can apply the RBF kernel to the same five points. The corresponding RBF kernel similarity

**FIGURE 8.6**

An example of five points.

matrix is

$$\begin{bmatrix} 0 & e^{-1} & e^{-1} & e^{-1} & e^{-1} \\ e^{-1} & 0 & e^{-2} & e^{-4} & e^{-2} \\ e^{-1} & e^{-2} & 0 & e^{-2} & e^{-4} \\ e^{-1} & e^{-4} & e^{-2} & 0 & e^{-2} \\ e^{-1} & e^{-2} & e^{-4} & e^{-2} & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0.37 & 0.37 & 0.37 & 0.37 \\ 0.37 & 1 & 0.135 & 0.02 & 0.135 \\ 0.37 & 0.135 & 1 & 0.135 & 0.02 \\ 0.37 & 0.02 & 0.135 & 1 & 0.135 \\ 0.37 & 0.135 & 0.02 & 0.135 & 1 \end{bmatrix}.$$

The magic here is that the RBF kernel indeed reduces the similarity between two points in a super-linear manner as the Euclidean distance between them increases. This nonlinear allocation of similarity enables k -means to assemble clusters using points that are not linearly separable and form clusters that are not convex. For example, if we apply the RBF kernel and k -means on the data set in Fig. 8.5(a), the output is Fig. 8.5(c), where the points in red (gray in print version) form a cluster and the points in blue (dark gray in print version) form another cluster. The output matches the visual intuition nicely.

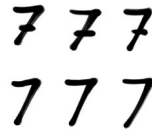
8.3 Hierarchical methods

While partitioning methods meet the basic clustering requirement of organizing a set of objects into a number of exclusive groups, in some situations we may want to partition our data into groups at different levels, or in general a hierarchy. A **hierarchical clustering method** works by grouping data objects into a hierarchy or “tree” of clusters.

In this section, you will study hierarchical clustering methods. Section 8.3.1 begins with a discussion about the basic concepts of hierarchical clustering. Then, Section 8.3.2 introduces the agglomerative, bottom-up approaches for hierarchical clustering. Section 8.3.3 presents the divisive, top-down approaches. Hierarchical clustering methods may be combined with other methods. Section 8.3.4 discusses BIRCH, a scalable hierarchical clustering method for large amounts of numeric data. Last, Section 8.3.5 describes the probabilistic hierarchical clustering methods.

8.3.1 Basic concepts of hierarchical clustering

Representing data objects in the form of a hierarchy is useful for data summarization and visualization. For example, as a manager of human resources in a company, you may organize your employees

**FIGURE 8.7**

Two clusters of handwritten digit 7s.

into major groups such as executives, managers, and staff. You can further partition these groups into smaller subgroups. For instance, the general group of staff can be further divided into subgroups of senior officers, officers, and trainees. All these groups form a hierarchy. We can easily summarize or characterize the data that is organized into a hierarchy, which can be used to find, say, the average salary of managers and of officers.

Consider handwritten character recognition as another example. A set of handwriting samples may be first partitioned into general groups where each group corresponds to a unique character. Some groups can be further partitioned into subgroups since a character may be written in multiple substantially different ways. For example, Fig. 8.7 shows a group of handwritten digit 7s. The group can be further divided into two subgroups, the first row being a subgroup where a short horizontal line is used in each writing, and the second row being another subgroup. If necessary, the hierarchical partitioning can be continued recursively until a desired granularity is reached.

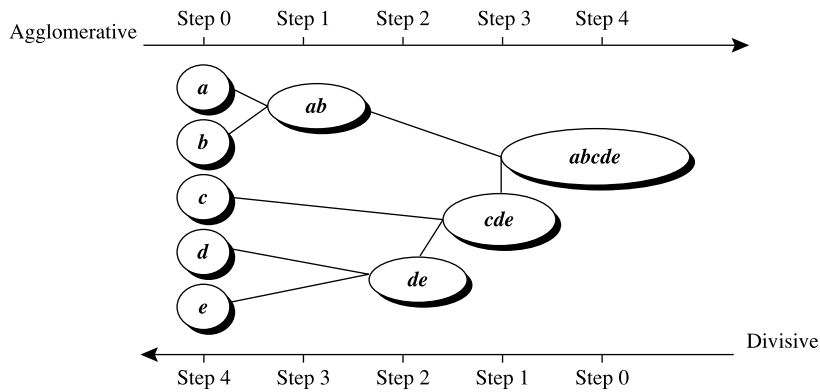
In the previous examples, although we partition the data hierarchically, we do not assume that the data has a hierarchical structure. Our use of a hierarchy here is just to summarize and represent the underlying data in a compressed way. Such a hierarchy is particularly useful for data visualization.

Alternatively, in some applications we may believe that the data bear an underlying hierarchical structure that we want to discover. For example, hierarchical clustering may uncover a hierarchy for the employees in a company structured on, say, salary. In the study of biological evolution, hierarchical clustering may group living creatures according to their biological features to uncover evolutionary paths, which are a hierarchy of species. As another example, grouping configurations of a strategic game (e.g., chess or checkers) in a hierarchical way may help to develop game strategies that can be used to train players.

A hierarchical clustering method can be either *agglomerative* or *divisive*, depending on whether the hierarchical decomposition is formed in a bottom-up (merging) or top-down (splitting) fashion. Let us have a closer look at these strategies.

An **agglomerative hierarchical clustering method** uses a bottom-up strategy. It typically starts by letting each object form its own cluster and iteratively merges clusters into larger and larger clusters, until all the objects are in a single cluster or certain termination conditions are satisfied. The single cluster becomes the hierarchy's root. For the merging step, it finds the two clusters that are closest to each other (according to some similarity measure) and combines the two to form one cluster. Because two clusters are merged per iteration, where each cluster contains at least one object, an agglomerative method requires at most n iterations.

A **divisive hierarchical clustering method** employs a top-down strategy. It starts by placing all objects in one cluster, which is the hierarchy's root. It then divides the root cluster into several smaller subclusters and recursively partitions those clusters into smaller ones. The partitioning process contin-

**FIGURE 8.8**

Agglomerative and divisive hierarchical clustering on data objects $\{a, b, c, d, e\}$.

ues until each cluster at the lowest level is coherent enough—either containing only one object, or the objects within a cluster are sufficiently similar to each other.

In either agglomerative or divisive hierarchical clustering, a user can specify the desired number of clusters as a termination condition.

Example 8.3. Agglomerative vs. divisive hierarchical clustering. Fig. 8.8 shows the application of an agglomerative hierarchical clustering method and a divisive hierarchical clustering method on a data set of five objects, $\{a, b, c, d, e\}$. Initially, the agglomerative method places each object into a cluster of its own. The clusters are then merged step-by-step according to some criterion. For example, clusters C_1 and C_2 may be merged if an object in C_1 and an object in C_2 form the minimum Euclidean distance between any two objects from different clusters. This is a **single-linkage** approach in that each cluster is represented by all the objects in the cluster, and the similarity between two clusters is measured by the similarity of the *closest* pair of data points belonging to different clusters. The cluster-merging process repeats until all the objects are eventually merged to form one cluster.

The divisive method proceeds in the contrasting way. All the objects are used to form one initial cluster. The cluster is split according to some principle such as the maximum Euclidean distance between the closest neighboring objects in the cluster. The cluster-splitting process repeats until, eventually, each new cluster contains only a single object. $\square$

The selection of merge or split points is critical for hierarchical clustering methods, because once a group of objects is merged or split, the process at the next step will operate on the newly generated clusters. It will neither undo what was done previously, nor perform object swapping between clusters. Thus merge or split decisions, if not well chosen, may lead to low-quality clusters. Moreover, the methods do not scale well because each decision of merge or split needs to examine and evaluate many objects or clusters.

A promising direction for improving the clustering quality of hierarchical methods is to integrate hierarchical clustering with other clustering techniques, resulting in **multiple-phase clustering**. We introduce BIRCH as a representative method in Section 8.3.4. BIRCH begins by partitioning objects

hierarchically using tree structures, where the leaf or low-level nonleaf nodes can be viewed as “microclusters” depending on the resolution scale. It then applies other clustering algorithms to perform macroclustering on the microclusters.

There are several orthogonal ways to categorize hierarchical clustering methods. For instance, they may be categorized into *deterministic* methods and *probabilistic* methods. Agglomerative, divisive, and multiphase methods are *deterministic*, since they consider data objects as deterministic and compute clusters according to the deterministic distances between objects. Probabilistic methods use probabilistic models to capture clusters and measure the quality of clusters by the fitness of models. We discuss probabilistic hierarchical clustering in Section 8.3.5.

8.3.2 Agglomerative hierarchical clustering

In this section, we discuss some important issues in agglomerative hierarchical clustering methods.

Similarity measures in hierarchical clustering

How can we choose which objects and clusters to merge in an agglomerative step? The core is to measure the similarity between two clusters, where each cluster is generally a set of objects.

Four widely used measures for distance between clusters are as follows, where $\|\mathbf{p} - \mathbf{p}'\|$ is the distance between two objects or points, $\mathbf{p}$ and $\mathbf{p}'$; $\mathbf{m}_i$ is the mean for cluster C_i ; and n_i is the number of objects in C_i . They are also known as *linkage measures*.

$$\textbf{Minimum distance: } dist_{min}(C_i, C_j) = \min_{\mathbf{p} \in C_i, \mathbf{p}' \in C_j} \{\|\mathbf{p} - \mathbf{p}'\|\} \quad (8.6)$$

$$\textbf{Maximum distance: } dist_{max}(C_i, C_j) = \max_{\mathbf{p} \in C_i, \mathbf{p}' \in C_j} \{\|\mathbf{p} - \mathbf{p}'\|\} \quad (8.7)$$

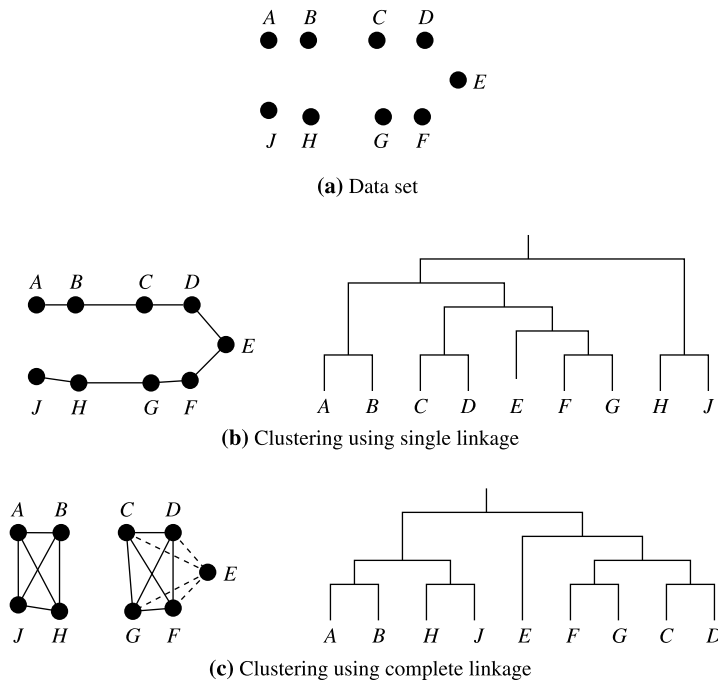
$$\textbf{Mean distance: } dist_{mean}(C_i, C_j) = \|\mathbf{m}_i - \mathbf{m}_j\| \quad (8.8)$$

$$\textbf{Average distance: } dist_{avg}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{\mathbf{p} \in C_i, \mathbf{p}' \in C_j} \|\mathbf{p} - \mathbf{p}'\| \quad (8.9)$$

When an algorithm uses the *minimum distance*, $d_{min}(C_i, C_j)$, to measure the distance between clusters, it is sometimes called a **nearest-neighbor clustering algorithm** or **single-linkage algorithm**. If we view the data points as nodes of a graph, with edges forming a path between the nodes in a cluster, then the merging of two clusters, C_i and C_j , corresponds to adding an edge between the nearest pair of nodes in C_i and C_j .

When an algorithm uses the *maximum distance*, $d_{max}(C_i, C_j)$, to measure the distance between clusters, it is sometimes called a **farthest-neighbor clustering algorithm** or **complete-linkage algorithm**. By viewing data points as nodes of a graph, with edges linking nodes, we can think of each cluster as a *complete* subgraph, that is, with edges connecting all the nodes in the clusters. The distance between two clusters is determined by the most distant nodes in the two clusters.

Similar to the situation in the k -means method, single-linkage and complete-linkage methods are sensitive to outliers. The use of *mean* or *average distance* is a compromise between the minimum and maximum distances and overcomes the outlier sensitivity problem. Whereas the *mean distance* is the simplest to compute, the *average distance* is advantageous in that it can handle categoric data and

**FIGURE 8.9**

Hierarchical clustering using single and complete linkages.

numeric data. The computation of the mean vector for categorical data can be difficult or impossible to define.

Example 8.4. Single vs. complete linkages. Let us apply hierarchical clustering to the data set of Fig. 8.9(a). Fig. 8.9(b) shows the hierarchy of clusters using single-linkage. Fig. 8.9(c) shows the case using complete linkage, where the edges between clusters $\{A, B, J, H\}$ and $\{C, D, G, F, E\}$ are omitted for ease of presentation. This example shows that by using single linkages we can find hierarchical clusters defined by local proximity, whereas complete linkage tends to find clusters opting for global closeness. $\square$

There are variations of the four essential linkage measures just discussed. For example, we can measure the distance between two clusters by the distance between the centroids (i.e., the central objects) of the clusters.

Connecting agglomerative hierarchical clustering and partitioning methods

Are there any connections between (agglomerative) hierarchical clustering and partitioning methods (Section 8.2)? Partitioning methods use the sum of squared errors (SSE) (Eq. (8.1)) to measure the compactness and quality of a possible clustering, that is a partitioning of points into clusters. Heuristically,

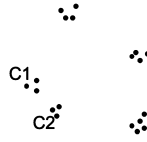


FIGURE 8.10

Ward's criterion.

agglomerative hierarchical clustering methods may also use the sum of squared errors (SSE) to guide the selection of clusters to merge.

For a data set of n points, if we set the number of clusters to n , a partitioning method naturally assigns each point into a cluster. This corresponds to the starting point of agglomerative hierarchical clustering. When we merge clusters in agglomerative clustering, we reduce the number of clusters. Which two clusters should we choose to merge? Heuristically, we may want to merge two clusters so that the resulting clustering also minimizes the sum of squared errors (SSE) (Eq. (8.1)), which is used as the criterion in partitioning methods like k -means.

Consider the five clusters in Fig. 8.10 as an illustrative example. Suppose we want to merge two of them into one so that we can have a hierarchy of clusters. Among all the possible pairs of clusters, merging C_1 and C_2 minimizes the SSE, and thus C_1 and C_2 should be merged in the next step in building the hierarchy.

The above intuition connecting agglomerative hierarchical clustering and partitioning methods gives us an alternative way to measure the similarity between two clusters. We can look at the increase of the SSE (Eq. (8.1)) if the two clusters are merged into one, the smaller the better. This is formulated by J.H. Ward and thus is known as **Ward's criterion**.

Suppose two disjoint clusters C_i and C_j are merged, and $m_{(ij)}$ is the mean of the new cluster. Then, Ward's criterion is defined as

$$\begin{aligned} W(C_i, C_j) &= \sum_{\mathbf{x} \in C_i \cup C_j} \|\mathbf{x} - \mathbf{m}_{(ij)}\|^2 - \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|^2 - \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \mathbf{m}_j\|^2 \\ &= \frac{n_i n_j}{n_i + n_j} \|\mathbf{m}_i - \mathbf{m}_j\|^2. \end{aligned}$$

The Lance-Williams algorithm

We discuss a few different proximity measures for clusters in agglomerative hierarchical clustering, is there a way to generalize them? Indeed, the **Lance-Williams formula** generalizes different measures in a uniform way. Suppose two exclusive clusters C_i and C_j are merged. We need to specify the distance between the merged cluster, denoted by $C_{(ij)}$, and every other cluster C_k . The similarity between the merged clusters $C_{(ij)}$ and cluster C_k is given by

$$d(C_{(ij)}, C_k) = \alpha_i d(C_i, C_k) + \alpha_j d(C_j, C_k) + \beta d(C_i, C_j) + \gamma |d(C_i, C_k) - d(C_j, C_k)|,$$

where α_i , α_j , β , and γ are parameters that together with the similarity function $d(C_i, C_j)$ determine the hierarchical clustering algorithm. As shown in the formula, the similarity between $C_{(ij)}$ and C_k is decided by four terms. The first two terms are the similarities between C_i and C_j to C_k , respectively.

The third term relies on the similarity between C_i and C_j . The last term represents how the difference of the original similarities between C_i and C_j to C_k may contribute to the new similarity.

For example, in the exercise, you are asked to verify that the single-linkage method is equivalent to taking $\alpha_i = \alpha_j = 0.5$, $\beta = 0$, and $\gamma = -0.5$ in the Lance-Williams formula. Record that in the single link method, the similarity between two clusters is determined by the similarity between of the closest pair of data points belonging to different clusters. Thus the above parameters simply pick the smaller one between $d(C_i, C_k)$ and $d(C_j, C_k)$. You can also verify that the complete-linkage method is equivalent to $\alpha_i = \alpha_j = 0.5$, $\beta = 0$, and $\gamma = 0.5$. Moreover, to implement the Ward's criterion, we can take $\alpha_i = \frac{n_i+n_k}{n_i+n_j+n_k}$, $\alpha_j = \frac{n_j+n_k}{n_i+n_j+n_k}$, $\beta = -\frac{n_k}{n_i+n_j+n_k}$, and $\gamma = 0$ in the Lance-Williams formula.

Using the Lance-Williams formula, the *Lance-Williams algorithm* generalizes agglomerative hierarchical clustering. It takes an agglomerative way and minimizes the sum of distance in each iteration until all points are merged into one cluster.

8.3.3 Divisive hierarchical clustering

A divisive hierarchical clustering method partitions a set of objects step by step into clusters. To design a divisive hierarchical clustering method, there are three important issues to consider.

First, a set of objects can be split in many different ways. A splitting criterion is needed to determine which splitting is the best. Technically, given two splittings, the splitting criterion should be able to tell which one is better. For example, the SSE (Eq. (8.1)) may be used on numeric data. If two splittings have the same number of clusters, then the one with a less SSE value is preferred. On nominal data, the Gini index (Chapter 6) may be used. The choice of splitting criteria is one of the most important decisions in designing a divisive hierarchical clustering method, since it determines what clustering results that the method may lead to.

Second, when we decide to split a cluster, we need to design a splitting method. While in general the splitting method should optimize the splitting criterion, one major consideration is the computational cost. For example, enumerating all possible splittings and finding the best one is likely computationally prohibitive. Thus some heuristic or approximate methods, such as bisecting k -means, that is, setting $k = 2$, may be used.

Third, in the middle of divisive hierarchical clustering, there are in general multiple clusters. Then, which cluster should be split next? An intuitive idea is to choose the “loosest” cluster. More concretely, we may compute the average SSE of each cluster, $E_{C_i} = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} (\mathbf{x} - \mathbf{m}_i)^2$, and choose the one with the largest average SSE to split.

The minimum spanning tree-based approach

Let us use the minimum spanning tree-based approach to illustrate the basic idea of divisive hierarchical clustering. In a weighted graph G , a minimum spanning tree is an acyclic subgraph that contains all nodes in G , and the sum of edge weights of the tree is minimized. For example, consider the weighted graph in Fig. 8.11, where the edges of weights up to nine are shown in the figure, and the edges of weights over nine are omitted. The minimum spanning tree consists of the edges in solid lines, whereas the edges in dashed lines and those omitted are not included in the minimum spanning tree. Minimum spanning tree can be computed using, for example, Prim's algorithm and Kruskal's algorithm.

Given a set of points, we can construct a weighted graph such that each point is represented by a node in the graph, and the distance between two points is the weight of the edge connecting the two

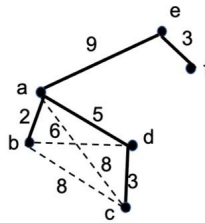


FIGURE 8.11

A weighted graph and a minimum spanning tree.

corresponding nodes in the graph. Then, we can compute the minimum spanning tree of the weighted graph. Intuitively, the minimum spanning tree can be regarded as the most compact way that the points are connected into one cluster. In general, in the process of divisive hierarchical clustering based on minimum spanning tree, every cluster is a subset of nodes and is represented by the minimum spanning tree, which is a subtree of the minimum spanning tree of the whole data set. The splitting criterion is the total weights of all the edges in the spanning tree(s) of all the clusters, the smaller the better.

Based on this spanning tree, we can progressively divide the set of points in one cluster into smaller clusters. Suppose we want to conduct bisecting splitting, that is, every time we split one cluster into two smaller clusters. At each step, we consider all edges in the spanning trees of the current clusters and delete the edge of the largest weight. Deleting an edge in a tree divides one cluster into two. Thus the splitting method is to divide a cluster by deleting the edge in the minimum spanning tree of the largest weight.

For example, consider a set of points $\{a, b, c, d, e, f\}$ as shown in Fig. 8.11. A weighted edge and the corresponding distance are plotted in the figure if the distance between two points is smaller than 10. Based on the minimum spanning tree method, the edge (a, e) , which has the largest weight in the minimum spanning tree, is first deleted, which divides the data set into two clusters, $\{a, b, c, d\}$ and $\{e, f\}$. Next, by deleting edge (a, d) , which has the largest weight in the remaining minimum spanning trees, the cluster $\{a, b, c, d\}$ is split into two smaller clusters $\{a, b\}$ and $\{c, d\}$. The process continues until each cluster has only one point and all the edges in the minimum spanning tree are deleted.

Dendrogram

A tree structure called a **dendrogram** is commonly used to represent the process of hierarchical clustering. It shows how objects are grouped together (in an agglomerative method) or partitioned (in a divisive method) step-by-step. Fig. 8.12 shows a dendrogram for the five objects presented in Fig. 8.8, where $l = 0$ shows the five objects as singleton clusters at level 0. At $l = 1$, objects a and b are grouped together to form the first cluster, and they stay together at all subsequent levels. We can also use a vertical axis to show the similarity scale between clusters. For example, when the similarity of two groups of objects, $\{a, b\}$ and $\{c, d, e\}$, is roughly 0.16, they are merged together to form a single cluster.

A complete dendrogram shows every data point as a node at the leaf level and the whole data set as one cluster at the root. In practice, however, too small clusters may not be very meaningful and too many such small clusters can be overwhelming. Thus a data analyst often shows and considers only the portion close to the root of a dendrogram. Moreover, when the graph contains a sufficiently small

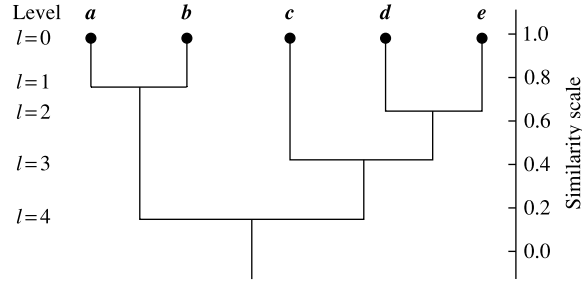


FIGURE 8.12

Dendrogram representation for hierarchical clustering of data objects $\{a, b, c, d, e\}$.

number of clusters, further merging them into even bigger ones may counter the objective of clustering analysis. Therefore the very root part of a dendrogram may also be ignored in analysis.

8.3.4 BIRCH: scalable hierarchical clustering using clustering feature trees

There are two major difficulties in the agglomerative and divisive hierarchical clustering methods discussed so far. First, all those methods cannot revisit any merge or split decisions made before. Thus an improper decision based on limited information may lead to low quality final clustering results. Moreover, scalability is a major bottleneck, since each merge or split needs to examine many possible options. To overcome those difficulties, we may conduct hierarchical clustering in multiple phases, so that clustering results can be improved over phases. Balanced Iterative Reducing and Clustering using Hierarchies (BIRCH) is such a method and is designed for clustering a large amount of numeric data by integrating hierarchical clustering (at the initial *microclustering* stage) and other clustering methods such as iterative partitioning (at the later *macroclustering* stage).

BIRCH uses the notions of *clustering feature* to summarize a cluster, and *clustering feature tree* (*CF-tree*) to represent a cluster hierarchy. These structures help the clustering method achieve good speed and scalability in large or even streaming databases and also make it effective for incremental and dynamic clustering of incoming objects.

Consider a cluster of n d -D data objects or points. The **clustering feature (CF)** of the cluster is a 3-D vector summarizing information about clusters of objects. It is defined as

$$CF = \langle n, LS, SS \rangle, \quad (8.10)$$

where LS is the linear sum of the n points (i.e., $LS = \sum_{i=1}^n \mathbf{x}_i$) and SS is the square sum of the data points (i.e., $SS = \sum_{i=1}^n \|\mathbf{x}_i\|^2$).

A clustering feature is essentially a summary of the statistics for the given cluster. Using a clustering feature, we can easily derive many useful statistics of a cluster. For example, the cluster's centroid, $\mathbf{x}_0$, radius, R , and diameter, D , are

$$\mathbf{x}_0 = \frac{\sum_{i=1}^n \mathbf{x}_i}{n} = \frac{LS}{n}, \quad (8.11)$$

$$R = \sqrt{\frac{\sum_{i=1}^n (\mathbf{x}_i - \mathbf{x}_0)^2}{n}} = \sqrt{\frac{SS}{n} - \left(\frac{\|\mathbf{LS}\|}{n}\right)^2}, \quad (8.12)$$

$$D = \sqrt{\frac{\sum_{i=1}^n \sum_{j=1}^n (\mathbf{x}_i - \mathbf{x}_j)^2}{n(n-1)}} = \sqrt{\frac{2nSS - 2\|\mathbf{LS}\|^2}{n(n-1)}}. \quad (8.13)$$

Here, R is the average distance from member objects to the centroid, and D is the average pairwise distance within a cluster. Both R and D reflect the tightness of the cluster around the centroid.

Summarizing a cluster using the clustering feature can avoid storing the detailed information about individual objects or points. Instead, we only need a constant size of space to store the clustering feature. This is the key to the efficiency of BIRCH in space. Moreover, clustering features are *additive*. That is, for two disjoint clusters, C_1 and C_2 , with the clustering features $\mathbf{CF}_1 = \langle n_1, \mathbf{LS}_1, SS_1 \rangle$ and $\mathbf{CF}_2 = \langle n_2, \mathbf{LS}_2, SS_2 \rangle$, respectively, the clustering feature for the cluster that formed by merging C_1 and C_2 is simply

$$\mathbf{CF}_1 + \mathbf{CF}_2 = \langle n_1 + n_2, \mathbf{LS}_1 + \mathbf{LS}_2, SS_1 + SS_2 \rangle. \quad (8.14)$$

Example 8.5. Clustering feature. Suppose there are three points, (2, 5), (3, 2), and (4, 3), in a cluster, C_1 . The clustering feature of C_1 is

$$\mathbf{CF}_1 = \langle 3, (2 + 3 + 4, 5 + 2 + 3), (2^2 + 3^2 + 4^2) + (5^2 + 2^2 + 3^2) \rangle = \langle 3, (9, 10), 67 \rangle.$$

Suppose that C_1 is disjoint to a second cluster, C_2 , where $\mathbf{CF}_2 = \langle 3, (35, 36), 857 \rangle$. The clustering feature of a new cluster, C_3 , that is formed by merging C_1 and C_2 , is derived by adding $\mathbf{CF}_1$ and $\mathbf{CF}_2$. That is,

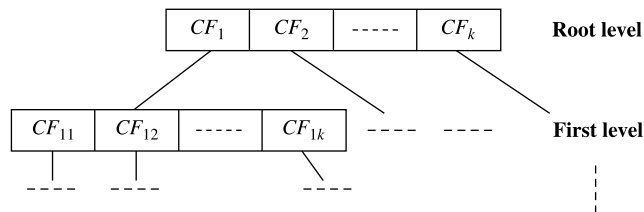
$$\mathbf{CF}_3 = \langle 3 + 3, (9 + 35, 10 + 36), 67 + 857 \rangle = \langle 6, (44, 46), 924 \rangle.$$

□

A **CF-tree** is a height-balanced tree that stores the clustering features for a hierarchical clustering. An example is shown in Fig. 8.13. By definition, a nonleaf node in a tree has descendants or “children.” The nonleaf nodes store sums of the CFs of their children and thus summarize clustering information about their children. A CF-tree has two parameters: *branching factor*, B , and *threshold*, T . The branching factor specifies the maximum number of children per nonleaf node. The threshold parameter specifies the maximum diameter of subclusters stored at the leaf nodes of the tree. These two parameters implicitly control the size of the resulting CF-tree.

Given a limited amount of main memory, an important consideration in BIRCH is to minimize the time required for input/output (I/O). BIRCH applies a *multiphase* clustering technique: A single scan of the data set yields a basic, good clustering, and one or more additional scans can optionally be used to further improve the quality. The primary phases are as follows:

- **Phase 1:** BIRCH scans the database to build an initial in-memory CF-tree, which can be viewed as a multilevel compression of the data that tries to preserve the inherent clustering structure of the data.

**FIGURE 8.13**

CF-tree structure.

- **Phase 2:** BIRCH applies a (selected) clustering algorithm to cluster the leaf nodes of the CF-tree, which removes sparse clusters as outliers and groups dense clusters into larger ones.

For Phase 1, the CF-tree is built dynamically as objects are inserted. Thus the method is incremental. An object is inserted into the closest leaf entry (subcluster). If the diameter of the subcluster stored in the leaf node after insertion is larger than the threshold value, then the leaf node and possibly other nodes are split. After the insertion of the new object, information about the object is passed toward the root of the tree. The size of the CF-tree can be changed by modifying the threshold. If the size of the memory that is needed for storing the CF-tree is larger than the size of the main memory available, then a larger threshold value can be specified and the CF-tree is rebuilt.

The rebuild process is performed by building a new tree from the leaf nodes of the old tree. Thus the process of rebuilding the tree is done without the necessity of rereading all the objects or points. This is similar to the insertion and node split in the construction of B+-trees. Therefore for building the tree, data have to be read just once. Some heuristics and methods have been introduced to deal with outliers and improve the quality of CF-trees by additional scans of the data. Once the CF-tree is built, any clustering algorithm, such as a typical partitioning algorithm, can be used with the CF-tree in Phase 2.

“How effective is BIRCH?” The time complexity of the algorithm is $O(n)$, where n is the number of objects to be clustered. Experiments have shown the linear scalability of the algorithm with respect to the number of objects, and good quality of clustering of data. However, since each node in a CF-tree can hold only a limited number of entries due to its size, a CF-tree node does not always correspond to what a user may consider a natural cluster. Moreover, if the clusters are not spherical in shape, BIRCH does not perform well because it uses the notion of radius or diameter to control the boundary of a cluster.

The ideas of clustering features and CF-trees have been applied beyond BIRCH. The ideas have been borrowed by many others to tackle problems of clustering streaming and dynamic data.

8.3.5 Probabilistic hierarchical clustering

Hierarchical clustering methods using linkage measures tend to be easy to understand and are often efficient in clustering. They are commonly used in many clustering analysis applications. However, hierarchical clustering methods can suffer from several drawbacks. First, choosing a good distance measure for hierarchical clustering is often far from trivial. Second, to apply such a method, the data

objects cannot have any missing attribute values. In the case where data is partially observed (i.e., some attribute values of some objects are missing), it is not easy to apply a hierarchical clustering method because the distance computation cannot be conducted. Third, most of the hierarchical clustering methods are heuristic and search locally at each step for a good merging/splitting decision. Consequently, the optimization goal of the resulting cluster hierarchy can be unclear.

Probabilistic hierarchical clustering aims to overcome some of these disadvantages by using probabilistic models to measure distances between clusters.

One way to look at the clustering problem is to regard the set of data objects to be clustered as a sample of the underlying data generation mechanism to be analyzed or, formally, the *generative model*. For example, when we conduct clustering analysis on a set of marketing surveys, we assume that the surveys collected are a sample of the opinions of all possible customers. Here, the data generation mechanism is a probability distribution of opinions with respect to different customers, which cannot be obtained directly and completely. The task of clustering is to estimate the generative model as accurately as possible using the observed data objects to be clustered.

In practice, we can assume that the data generative models adopt common distribution functions, such as Gaussian distribution or Bernoulli distribution, which are governed by parameters. The task of learning a generative model is then reduced to finding the parameter values for which the model best fits the observed data set.

Example 8.6. Generative model. Suppose we are given a set of 1-D points $X = \{x_1, \dots, x_n\}$ for clustering analysis. Let us assume that the data points are generated by a Gaussian distribution,

$$\mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad (8.15)$$

where the parameters are μ (the mean) and σ^2 (the variance).

The probability that a point $x_i \in X$ is then generated by the model is

$$P(x_i|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}. \quad (8.16)$$

Consequently, the likelihood that the data set X observed is generated by the model is

$$L(\mathcal{N}(\mu, \sigma^2) : X) = P(X|\mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}. \quad (8.17)$$

The task of learning the generative model is to find the parameters μ and σ^2 such that the likelihood $L(\mathcal{N}(\mu, \sigma^2) : X)$ is maximized, that is, finding

$$\mathcal{N}(\mu_0, \sigma_0^2) = \arg \max \{L(\mathcal{N}(\mu, \sigma^2) : X)\}, \quad (8.18)$$

where $\max\{L(\mathcal{N}(\mu, \sigma^2) : X)\}$ is called the *maximum likelihood*. $\square$

Given a set of objects, the quality of a cluster formed by all the objects can be measured by the maximum likelihood. For a set of objects partitioned into m clusters $C_1, \dots, C_m$. Then the quality can

be measured by

$$Q(\{C_1, \dots, C_m\}) = \prod_{i=1}^m P(C_i), \quad (8.19)$$

where $P()$ is the maximum likelihood. To calculate $P(C_i)$, we can fit each cluster C_i ($1 \leq i \leq m$) by a generative model M_i , and estimate the probability by $P(C_i) = \prod_{x \in C_i} P(x|M_i)$. If we merge two clusters, C_{j_1} and C_{j_2} , into a cluster, $C_{j_1} \cup C_{j_2}$, then, the change in quality of the overall clustering is

$$\begin{aligned} & Q(\{C_1, \dots, C_m\} - \{C_{j_1}, C_{j_2}\} \cup \{C_{j_1} \cup C_{j_2}\}) - Q(\{C_1, \dots, C_m\}) \\ &= \frac{\prod_{i=1}^m P(C_i) \cdot P(C_{j_1} \cup C_{j_2})}{P(C_{j_1})P(C_{j_2})} - \prod_{i=1}^m P(C_i) \\ &= \prod_{i=1}^m P(C_i) \left(\frac{P(C_{j_1} \cup C_{j_2})}{P(C_{j_1})P(C_{j_2})} - 1 \right). \end{aligned} \quad (8.20)$$

When choosing to merge two clusters in hierarchical clustering, $\prod_{i=1}^m P(C_i)$ is constant for any pair of clusters. Therefore given clusters C_1 and C_2 , the dissimilarity between them can be measured by

$$\text{dist}(C_1, C_2) = -\log \frac{P(C_1 \cup C_2)}{P(C_1)P(C_2)}. \quad (8.21)$$

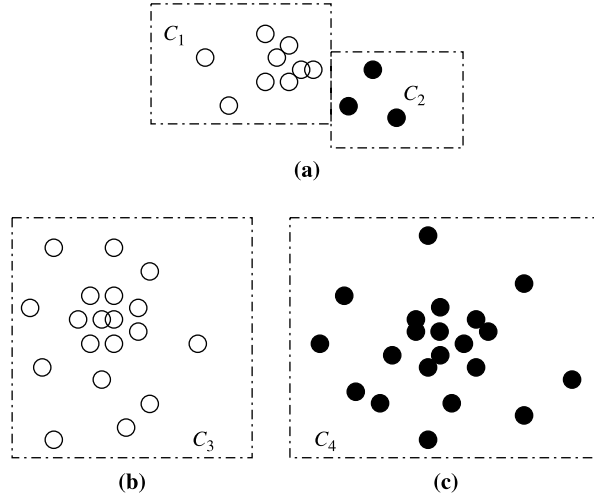
A probabilistic hierarchical clustering method can adopt the agglomerative clustering framework, but use probabilistic models (Eq. (8.21)) to measure the similarity between clusters.

Upon close observation of Eq. (8.20), we see that merging two clusters may not always lead to an improvement in clustering quality, that is, $\frac{P(C_{j_1} \cup C_{j_2})}{P(C_{j_1})P(C_{j_2})}$ may be less than 1. For example, assume that Gaussian distribution functions are used in the model of Fig. 8.14. Although merging clusters C_1 and C_2 results in a cluster that better fits a Gaussian distribution, merging clusters C_3 and C_4 lowers the clustering quality because no Gaussian functions can fit the merged cluster well.

Based on this observation, a probabilistic hierarchical clustering scheme can start with one cluster per object and merge two clusters, C_i and C_j , if the distance between them is negative. In each iteration, we try to find C_i and C_j so as to maximize $\log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)}$. The iteration continues as long as $\log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)} > 0$, that is, as long as there is an improvement in clustering quality. The pseudocode is given in Fig. 8.15.

Probabilistic hierarchical clustering methods are easy to understand and generally have the same efficiency as agglomerative hierarchical clustering methods; in fact, they share the same framework. Probabilistic models are more interpretable, but sometimes less flexible than distance metrics. Probabilistic models can handle partially observed data. For example, given a multidimensional data set where some objects have missing values on some dimensions, we can learn a Gaussian model on each dimension independently using the observed values on the dimension. The resulting cluster hierarchy accomplishes the optimization goal of fitting data to the selected probabilistic models.

A drawback of using probabilistic hierarchical clustering is that it outputs only one hierarchy with respect to a chosen probabilistic model. It cannot handle the uncertainty of cluster hierarchies. Given a

**FIGURE 8.14**

Merging clusters in probabilistic hierarchical clustering: (a) Merging clusters C_1 and C_2 leads to an increase in overall cluster quality, but merging clusters (b) C_3 and (c) C_4 does not.

Algorithm: A probabilistic hierarchical clustering algorithm.

Input:

- $D = \{o_1, \dots, o_n\}$: a data set containing n objects;

Output: A hierarchy of clusters.

Method:

- (1) **create** a cluster for each object $C_i = \{o_i\}$, $1 \leq i \leq n$;
- (2) **for** $i = 1$ to n
- (3) **find** pair of clusters C_i and C_j such that $C_i, C_j = \arg \max_{i \neq j} \log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)}$;
- (4) **if** $\log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)} > 0$ **then** merge C_i and C_j ;
- (5) **else** stop;

FIGURE 8.15

A probabilistic hierarchical clustering algorithm.

data set, there may exist multiple hierarchies that fit the observed data. Neither algorithmic approaches nor probabilistic approaches can find the distribution of such hierarchies. Recently, Bayesian tree-structured models have been developed to handle such problems. Bayesian and other sophisticated probabilistic clustering methods are considered advanced topics and are not covered in this book.

8.4 Density-based and grid-based methods

Most of the partitioning and hierarchical methods are designed to find spherical-shaped clusters. They have difficulty finding clusters of arbitrary shape such as the “S” shape and oval clusters in Fig. 8.16.

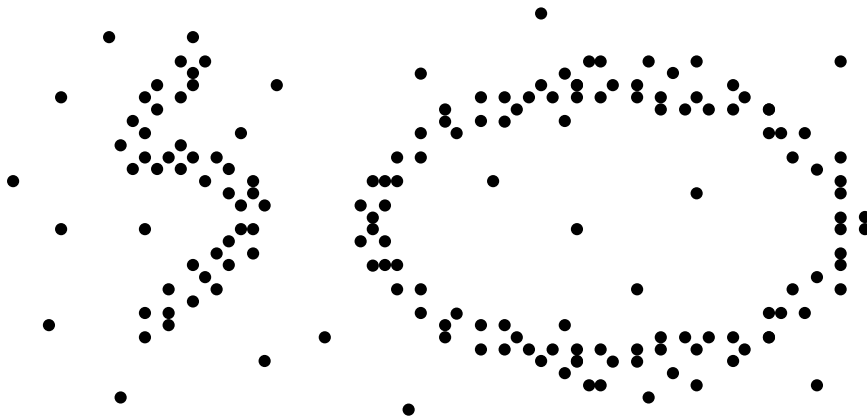


FIGURE 8.16

Clusters of arbitrary shape.

Although some feature transformation methods, such as kernel k -means, may help, it is often tricky to choose appropriate kernel functions. Given such data, they would likely inaccurately identify convex regions, where noises or outliers are included in the clusters.

To find clusters of arbitrary shape, alternatively, we can model clusters as dense regions in the data space, separated by sparse regions. This is the main strategy behind *density-based clustering methods*, which can discover clusters of nonspherical shape. In this section, you will learn the basic techniques of density-based clustering by studying two representative methods, namely, DBSCAN (Section 8.4.1) and DENCLUE (Section 8.4.2). To tackle the computational cost in density-based clustering, the data space may be partitioned into a grid. This idea motivates the grid-based clustering methods (Section 8.4.3).

8.4.1 DBSCAN: density-based clustering based on connected regions with high density

“How can we find dense regions in density-based clustering?” The *density* of an object o can be measured by the number of objects close to o . **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) finds *core objects*, that is, objects that have dense neighborhoods. It connects core objects and their neighborhoods to form dense regions as clusters. In this section, let us explain **DBSCAN***, an improved version of the original **DBSCAN**.

“How does **DBSCAN*** quantify the neighborhood of an object?” **DBSCAN*** employs a user-specified parameter $\epsilon > 0$ to specify the radius of a neighborhood that is considered for every object. The ϵ -**neighborhood** of an object o is the space within a radius ϵ centered at o .

Due to the fixed neighborhood size parameterized by ϵ , the **density of a neighborhood** can be measured simply by the number of objects in the neighborhood. To determine whether a neighborhood is dense or not, **DBSCAN*** uses another user-specified parameter, *Min Pts*, which specifies the density

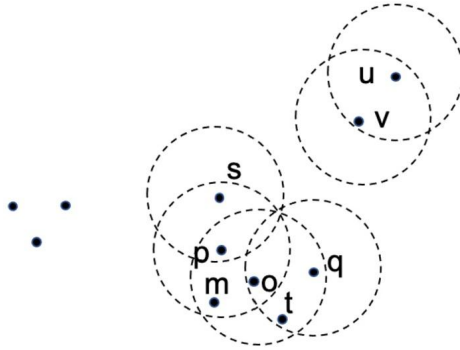


FIGURE 8.17

Density-reachability and density-connectivity in DBSCAN.

threshold of dense regions. An object is a **core object** if the ϵ -neighborhood of the object contains at least *MinPts* objects, otherwise, it is **noise**. Core objects are the pillars of dense regions.

Given a set, D , of objects, we can identify all core objects with respect to the given parameters, ϵ and *MinPts*. The clustering task is therein reduced to using core objects and their neighborhoods to form dense regions, where the dense regions are clusters.

Two core objects p and q are **ϵ -reachable** if $d(p, q) \leq \epsilon$, that is, p is in the ϵ -neighbor of q and vice versa. Two core objects p and q are **density-connected** if p and q are ϵ -reachable or transitively ϵ -reachable, where p and q are transitively ϵ -reachable if there exist one or multiple core objects $r_1, \dots, r_l$ such that p and r_1 are ϵ -reachable, r_i and r_{i+1} ($1 \leq i < l$) are ϵ -reachable, and r_l and q are ϵ -reachable. Then, a cluster C with respect to parameters ϵ and *MinPts* is simply a nonempty maximal subset of core objects that every pair of objects in C is density connected.

In DBSCAN*, a cluster contains only core objects. However, it is easy to assign a noncore object that is in the ϵ -neighborhood of a core object to the cluster containing that core object. DBSCAN explicitly identifies such noncore objects as **border objects**, and DBSCAN* can use a postprocessing step to pick up those border objects. Those objects that do not belong to any ϵ -neighborhood are outliers.

Example 8.7. Density-reachability and density-connectivity. Consider Fig. 8.17 for a given ϵ represented by the radius of the circles, and, say, let *MinPts* = 3.

Of the labeled objects, p, m, o, q , and t are core objects, since each of the ϵ -neighborhoods (dashed circles in the figure) of them contains at least three objects. Objects p and o are ϵ -reachable, so are o and q . Thus p and q are density-connected.

It can be verified that the core objects p, m, o, q , and t form a cluster, since each two among them are density-connected and no other core objects can be added into this group so that the pairwise density-connectivity is maintained.

Object s is not a core object, since the ϵ -neighborhood of s contains only two objects. However, s is in the ϵ -neighborhood of core object p , thus s is a border object.

Objects u and v are not core objects, and they do not belong to the ϵ -neighborhood of any core objects. Thus they are outliers. $\square$

Algorithm: DBSCAN*: a density-based clustering algorithm.

Input:

- D : a data set containing n objects,
- ϵ : the radius parameter, and
- $MinPts$: the neighborhood density threshold.

Output: A set of density-based clusters.

Method:

```

(1)  mark all objects as unvisited;
(2)  do
(3)      randomly select an unvisited object  $p$ ;
(4)      mark  $p$  as visited;
(5)      if the  $\epsilon$ -neighborhood of  $p$  has at least  $MinPts$  objects
(6)          create a new cluster  $C$ , and add  $p$  to  $C$ ;
(7)          let  $N$  be the set of objects in the  $\epsilon$ -neighborhood of  $p$ ;
(8)          for each point  $p'$  in  $N$ 
(9)              if  $p'$  is unvisited
(10)                  mark  $p'$  as visited;
(11)                  if the  $\epsilon$ -neighborhood of  $p'$  has at least  $MinPts$  points,
                      add those points to  $N$  and add  $p'$  to  $C$ ;
(12)          end for
(13)          output  $C$ ;
(14)      else mark  $p$  as noise;
(15)  until no object is unvisited;
```

FIGURE 8.18

DBSCAN* algorithm.

“How does DBSCAN find clusters?”* Initially, all objects in a given data set D are marked as “unvisited.” DBSCAN* randomly selects an unvisited object p , marks p as “visited,” and checks whether the ϵ -neighborhood of p contains at least $MinPts$ objects. If not, p is marked as a noise point. Otherwise, a new cluster C is created for p , and all the objects in the ϵ -neighborhood of p are added to a candidate set, N .

DBSCAN* iteratively adds to C those core objects in N that do not belong to any cluster. In this process, for an object p' in N that carries the label “unvisited,” DBSCAN* marks it as “visited” and checks its ϵ -neighborhood. If the ϵ -neighborhood of p' has at least $MinPts$ objects, p' is labeled as a core object and added into C , those objects in the ϵ -neighborhood of p' are added to N . DBSCAN* continues adding objects to C until C can no longer be expanded, that is, N is empty. At this time, cluster C is completed, and thus is output.

To find the next cluster, DBSCAN* randomly selects an unvisited object from the remaining ones. The clustering process continues until all objects are visited. The pseudocode of the DBSCAN* algorithm is given in Fig. 8.18.

If a spatial index is used, the computational complexity of DBSCAN* is $O(n \log n)$, where n is the number of database objects. Otherwise, the complexity is $O(n^2)$. With appropriate settings of the user-defined parameters, ϵ and $MinPts$, the algorithm is effective in finding arbitrary-shaped clusters.

It is not easy to specify two parameters, ϵ and $MinPts$, in DBSCAN. Moreover, density-based clusters may also have hierarchies. For example, within a dense area there may be a sub-area is sub-*

stantially denser. Can we find hierarchical density-based clusters? Indeed, DBSCAN* can be extended to HDBSCAN*, which achieves density-based hierarchical clustering.

HDBSCAN* only takes one parameter, *MinPts*. For an object p , the **core distance** of p , denoted by $d_{core}(p)$ is the distance from p to its *MinPts*th nearest neighbor (including p itself). In other words, $d_{core}(p)$ is the minimum radius with respect to which p is a core object in DBSCAN*. For two objects p and q , the **mutual reachability distance** between them is $d_{mreach}(p, q) = \max\{d_{core}(p), d_{core}(q), d(p, q)\}$. In other words, $d_{mreach}(p, q)$ is the minimum radius ϵ such that p and q are ϵ -reachable in DBSCAN*.

Given a set of objects as the input to HDBSCAN*, we can construct a **mutual reachability graph** G_{MinPts} , which is a complete graph. Every object in the input is a node in the mutual reachability graph. The weight of the edge between p and q is the mutual reachability distance $d_{mreach}(p, q)$. We can apply the minimum spanning tree approach (Section 8.3.3) on the mutual reachability graph to find density-based hierarchical clusters.

To further reduce the demand of setting parameters, a cluster analysis method called **OPTICS** was proposed. OPTICS does not explicitly produce a data set clustering. Instead, it outputs a **cluster ordering**, a linear list of all objects under analysis, representing the *density-based clustering structure* of the data. Objects in a denser cluster are listed closer to each other in the cluster ordering. This ordering is equivalent to density-based clustering obtained from a wide range of parameter settings. Thus, OPTICS does not require the user to provide a specific density threshold. The cluster ordering can be used to extract basic clustering information (e.g., cluster centers, or arbitrary-shaped clusters), derive the intrinsic clustering structure, and provide a visualization of the clustering.

To construct the different clusterings simultaneously, the objects are processed in a specific order. This order selects an object that is density-reachable with respect to the lowest ϵ value so that clusters with higher density (i.e., lower ϵ) will be finished first. For example, Fig. 8.19 shows the reachability plot for a simple 2-D data set, which presents a general overview of how the data are structured and clustered. The data objects are plotted in the clustering order (horizontal axis) together with their respective reachability-distances (vertical axis). The three Gaussian “bumps” in the plot reflect three clusters in the data set.

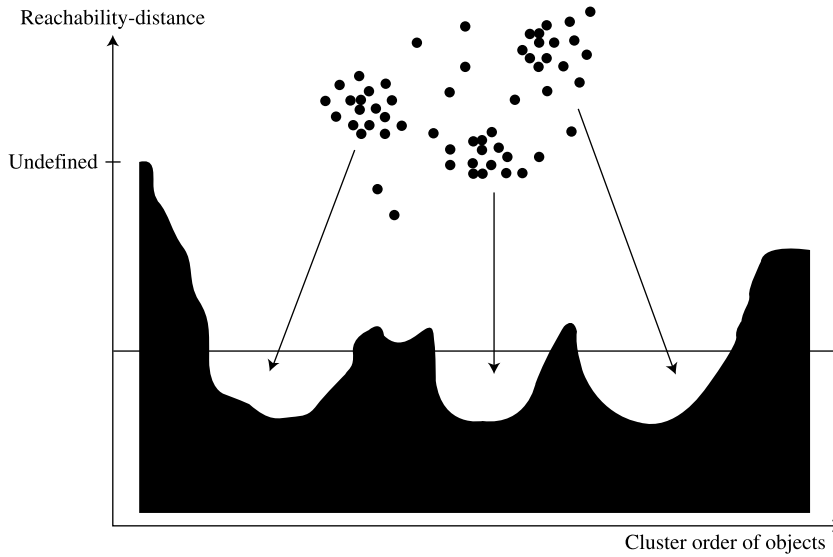
OPTICS can be seen as a generalization of DBSCAN that replaces the ϵ parameter with a maximum value that mostly affects performance. *MinPts* then essentially becomes the minimum cluster size to find. While the algorithm is much easier to parameterize than DBSCAN, it usually produces a hierarchical clustering instead of the simple data partitioning that DBSCAN produces.

8.4.2 DENCLUE: clustering based on density distribution functions

Density estimation is a core issue in density-based clustering. **DENCLUE** (DENSITY-based CLUstEring) is a clustering method based on a set of density distribution functions. We first give some background on density estimation and then describe the DENCLUE algorithm.

In probability and statistics, **density estimation** is the estimation of an unobservable underlying probability density function based on a set of observed data. In the context of density-based clustering, the unobservable underlying probability density function is the true distribution of the population of all possible objects to be analyzed. The observed data set is regarded as a random sample from that population.

In DENCLUE, **kernel density estimation** is used, which is a nonparametric density estimation approach from statistics. The general idea behind kernel density estimation is simple. We treat an observed

**FIGURE 8.19**

Cluster ordering in OPTICS. *Source:* Adapted from Ankerst, Breunig, Kriegel, and Sander [ABKS99].

object as an indicator of high-probability density in the surrounding region. The probability density at a point depends on the distances from this point to the observed objects.

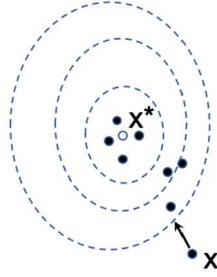
Formally, let $\mathbf{x}_1, \dots, \mathbf{x}_n$ be an independent and identically distributed sample of a random variable f . The *kernel density approximation of the probability density function* is

$$\hat{f}_h(\mathbf{x}) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right), \quad (8.22)$$

where $K()$ is a kernel function and h is the bandwidth serving as a smoothing parameter. A **kernel** can be regarded as a function modeling the influence of a sample point within its neighborhood. Technically, a kernel $K()$ is a nonnegative real-valued integrable function that should satisfy two requirements: $\int_{-\infty}^{+\infty} K(u)du = 1$ and $K(-u) = K(u)$ for all values of u . A frequently used kernel is a standard Gaussian function with a mean of 0 and a variance of 1:

$$K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(\mathbf{x} - \mathbf{x}_i)^2}{2h^2}}. \quad (8.23)$$

DENCLUE uses a Gaussian kernel to estimate density based on the given set of objects to be clustered. A point $\mathbf{x}^*$ is called a **density attractor** if it is a local maximum of the estimated density function. To avoid trivial local maximum points, DENCLUE uses a noise threshold, ξ , and only considers those density attractors $\mathbf{x}^*$ such that $\hat{f}(\mathbf{x}^*) \geq \xi$. These nontrivial density attractors are the centers of clusters.

**FIGURE 8.20**

Hill-climbing in DENCLUE.

The objects under analysis are assigned to clusters through density attractors using a stepwise hill-climbing procedure. For an object, $\mathbf{x}$, the hill-climbing procedure starts from $\mathbf{x}$ and is guided by the gradient of the estimated density function. That is, the density attractor for $\mathbf{x}$ is computed as

$$\begin{aligned} \mathbf{x}^0 &= \mathbf{x} \\ \mathbf{x}^{j+1} &= \mathbf{x}^j + \delta \frac{\nabla \hat{f}(\mathbf{x}^j)}{|\nabla \hat{f}(\mathbf{x}^j)|}, \end{aligned} \quad (8.24)$$

where δ is a parameter to control the speed of convergence, and

$$\nabla \hat{f}(\mathbf{x}) = \frac{1}{h^{d+2n} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right) (\mathbf{x}_i - \mathbf{x})}. \quad (8.25)$$

The hill-climbing procedure stops at step $k > 0$ if $\hat{f}(\mathbf{x}^{k+1}) < \hat{f}(\mathbf{x}^k)$, and assigns $\mathbf{x}$ to the density attractor $\mathbf{x}^* = \mathbf{x}^k$. An object $\mathbf{x}$ is an outlier or noise if it converges in the hill-climbing procedure to a local maximum $\mathbf{x}^*$ with $\hat{f}(\mathbf{x}^*) < \xi$.

Fig. 8.20 illustrates the hill-climbing idea. For a point $\mathbf{x}$, the density attractor for $\mathbf{x}$ is initialized to $\mathbf{x}^0 = \mathbf{x}$. In the next iteration, the density attractor moves a small step towards the direction indicated by the gradient of the density function, shown by the arrow in the figure, until the point $\mathbf{x}^*$ where density is stable (the white circle point in the figure), which is a local optimal.

A cluster in DENCLUE is a set of density attractors X and a set of input objects C such that each object in C is assigned to a density attractor in X , and there exists a path between every pair of density attractors where the density is above ξ . By using multiple density attractors connected by paths, DENCLUE can find clusters of arbitrary shape.

DENCLUE has several advantages. It can be regarded as a generalization of several well-known clustering methods such as single-linkage approaches and DBSCAN. Moreover, DENCLUE is invariant against noise. The kernel density estimation can effectively reduce the influence of noise by uniformly distributing noise into the input data.

8.4.3 Grid-based methods

As analyzed in the previous subsections, computing density for density-based clustering may be costly, particularly on large data sets and data sets of high dimensionality. To tackle the efficiency and scalability challenges, one idea is to partition the data space into cells using a grid. This motivates the grid-based clustering methods.

The *grid-based clustering* approach uses a multiresolution grid data structure. It quantizes the object space into a finite number of cells that form a grid structure on which all of the operations for clustering are performed. The main advantage of the approach is its fast processing time, which is typically independent of the number of data objects, yet dependent on only the number of cells in each dimension in the quantized space.

Typically, grid-based clustering takes three steps.

1. We create a grid structure so that the data space is partitioned into a finite number of cells.
2. For each cell, we calculate the cell density. By a carefully designed method, we may be able to scan the data once and derive the densities for all cells. This step is a key to gain efficiency and scalability.
3. We use the dense cells to assemble clusters and optionally summarize dense cells and the corresponding clusters.

Let us illustrate this using an example. **CLIQUE** (CLustering In QUES) is a simple grid-based method for finding density-based clusters in subspaces. CLIQUE partitions each dimension into nonoverlapping intervals, thereby partitioning the entire data space into cells. It uses a density threshold to identify *dense* cells and *sparse* ones. A cell is dense if the number of objects mapped to it exceeds the density threshold.

The main strategy behind CLIQUE for identifying a candidate search space uses the monotonicity of dense cells with respect to dimensionality. This is based on the *Apriori property* used in frequent pattern and association rule mining (Chapter 4). In the context of clusters in subspaces, the monotonicity says the following. A k -dimensional cell c ($k > 1$) can have at least l points only if every $(k - 1)$ -dimensional projection of c , which is a cell in a $(k - 1)$ -dimensional subspace, has at least l points. Consider Fig. 8.21, where the data space contains three dimensions: *age*, *salary*, and *vacation*. A 2-D

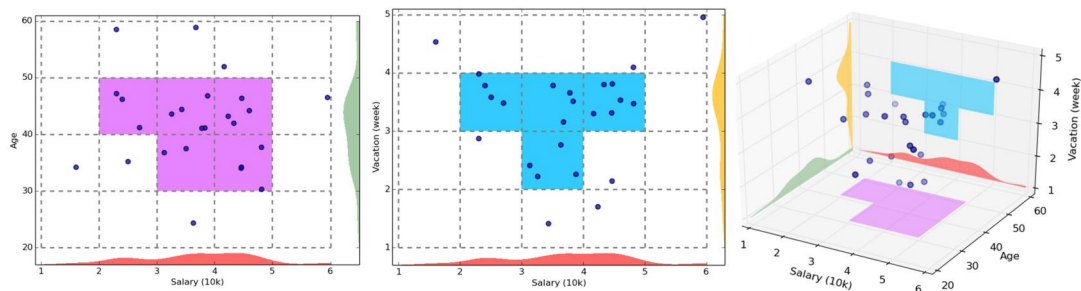


FIGURE 8.21

Dense units found with respect to *age* for the dimensions *salary* and *vacation* are intersected to provide a candidate search space for dense units of higher dimensionality.

cell, say in the subspace formed by *age* and *salary*, contains l points only if the projection of this cell in every dimension, that is, *age* and *salary*, respectively, contains at least l points.

CLIQUE performs clustering in three steps. In the first step, CLIQUE partitions the d -dimensional data space into nonoverlapping rectangular units.

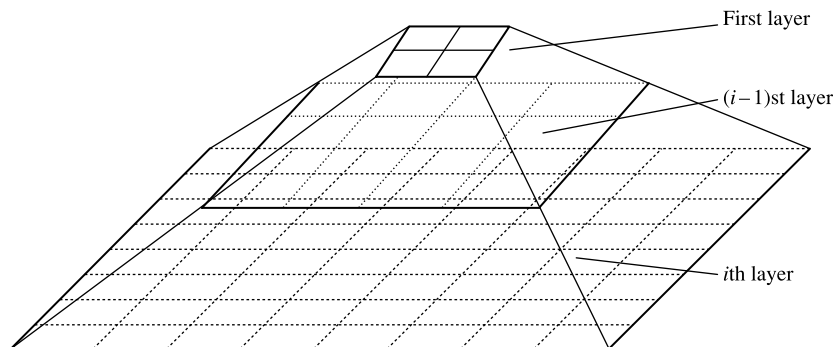
In the second step, CLIQUE identifies the dense units among these. CLIQUE finds dense cells in all of the subspaces. To do so, CLIQUE partitions every dimension into intervals, and identifies intervals containing at least l points, where l is the density threshold. CLIQUE then iteratively joins two k -dimensional dense cells, c_1 and c_2 , in subspaces $(D_{i_1}, \dots, D_{i_k})$ and $(D_{j_1}, \dots, D_{j_k})$, respectively, if $D_{i_1} = D_{j_1}, \dots, D_{i_{k-1}} = D_{j_{k-1}}$, and c_1 and c_2 share the same intervals in those dimensions. The join operation generates a new $(k + 1)$ -dimensional candidate cell c in space $(D_{i_1}, \dots, D_{i_{k-1}}, D_{i_k}, D_{j_k})$. CLIQUE checks whether the number of points in c passes the density threshold. The iteration terminates when no candidates can be generated or no candidate cells are dense.

In the last step, CLIQUE uses the dense cells in each subspace to assemble clusters, which can be of arbitrary shape. The idea is to apply the Minimum Description Length (MDL) principle (Chapter 7) to use the *maximal regions* to cover connected dense cells, where a maximal region is a hyperrectangle where every cell falling into this region is dense, and the region cannot be extended further in any dimension in the subspace. Finding the best description of a cluster in general is NP-hard. Thus CLIQUE adopts a simple greedy approach. It starts with an arbitrary dense cell, finds a maximal region covering the cell, and then works on the remaining dense cells that have not yet been covered. The greedy method terminates when all dense cells are covered.

“How effective is CLIQUE?” CLIQUE automatically finds subspaces of the highest dimensionality such that high-density clusters exist in those subspaces. It is insensitive to the order of input objects and does not presume any canonical data distribution. It scales linearly with the size of the input and has good scalability as the number of dimensions in the data increases. However, obtaining a meaningful clustering is dependent on proper tuning of the grid size (which is a stable structure here) and the density threshold. This can be difficult in practice because the grid size and density threshold are used across all combinations of dimensions in the data set. Thus the accuracy of the clustering results may be degraded at the expense of the method’s simplicity. Moreover, for a given dense region, all projections of the region onto lower-dimensionality subspaces will also be dense. This can result in a large overlap among the reported dense regions. Furthermore, it is difficult to find clusters of rather different densities within different dimensional subspaces.

STING is another representative grid-based multiresolution clustering technique. In STING, the spatial area of the input objects is divided into rectangular cells. The space can be divided in a hierarchical and recursive way. Several levels of such rectangular cells correspond to different levels of resolution and form a hierarchical structure: Each cell at a high level is partitioned to form a number of cells at the next lower level. Statistical information regarding the attributes in each grid cell, such as the mean, maximum, and minimum values, is precomputed and stored as *statistical parameters*. These statistical parameters are useful for query processing and for other data analysis tasks.

Fig. 8.22 shows a hierarchical structure for STING clustering. The statistical parameters of higher-level cells can easily be computed from the parameters of the lower-level cells. These parameters include the following: the attribute-independent parameter, *count*; and the attribute-dependent parameters, *mean*, *stdev* (standard deviation), *min* (minimum), *max* (maximum), and the type of *distribution* that the attribute value in the cell follows such as *normal*, *uniform*, *exponential*, or *none* (if the distribution is unknown). Here, the attribute is a selected measure for analysis such as *price* for house objects.

**FIGURE 8.22**

Hierarchical structure for STING clustering.

When the data are loaded into the database, the parameters *count*, *mean*, *stdev*, *min*, and *max* of the bottom-level cells are calculated directly from the data. The value of *distribution* may either be assigned by the user if the distribution type is known beforehand or obtained by hypothesis tests such as the χ^2 test. The type of distribution of a higher-level cell can be computed based on the majority of distribution types of its corresponding lower-level cells in conjunction with a threshold filtering process. If the distributions of the lower-level cells disagree with each other and fail the threshold test, the distribution type of the high-level cell is set to *none*.

“How is this statistical information useful for query answering?” The statistical parameters can be used in a top-down, grid-based manner as follows. First, a layer within the hierarchical structure is determined from which the query-answering process is to start. This layer typically contains a small number of cells. For each cell in the current layer, we compute the confidence interval (or estimated probability range) reflecting the relevancy of the cell to the given query. The irrelevant cells are removed from further consideration. Processing of the next lower level examines only the remaining relevant cells. This process repeats until the bottom layer is reached. At this time, if the query specification is met, the regions of relevant cells that satisfy the query are returned. Otherwise, the data points that fall into the relevant cells are retrieved and further processed until they meet the query’s requirements.

An interesting property of STING is that it approaches the clustering result of DBSCAN if the granularity approaches 0 (i.e., toward very low-level data). In other words, using the count and cell size information, dense clusters can be identified approximately using STING. Therefore STING can also be regarded as a density-based clustering method.

“What advantages does STING offer over other clustering methods?” STING offers several advantages. First, the grid-based computation is *query-independent* because the statistical information stored in each cell represents the summary information of the data in the grid cell, independent of the query. Moreover, the grid structure facilitates parallel processing and incremental updating. Last, the efficiency of STING is a major advantage: STING goes through the database once to compute the statistical parameters of the cells and hence the time complexity of generating clusters is $O(n)$, where n is the total number of objects. After generating the hierarchical structure, the query processing time

is $O(g)$, where g is the total number of grid cells at the lowest level, which is usually much smaller than n .

Because STING uses a multiresolution approach to cluster analysis, the quality of STING clustering depends on the granularity of the lowest level of the grid structure. If the granularity is very fine, the cost of processing increases substantially; however, if the bottom level of the grid structure is too coarse, it may reduce the quality of cluster analysis. Moreover, STING does not consider the spatial relationship between the children and their neighboring cells for construction of a parent cell. As a result, the shapes of the resulting clusters are isothetic, that is, all the cluster boundaries are either horizontal or vertical, and no diagonal boundary is detected. This may lower the quality and accuracy of the clusters despite the fast processing time of the technique.

8.5 Evaluation of clustering

By now you have learned what clustering is and know several popular clustering methods. You may ask, “*When I try out a clustering method on a data set, how can I evaluate whether the clustering results are good?*” In general, *cluster evaluation* assesses the feasibility of clustering analysis on a data set and the quality of the results generated by a clustering method. The major tasks of clustering evaluation include the following:

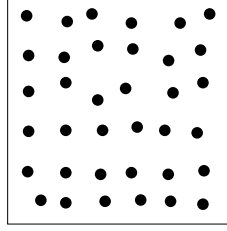
- *Assessing clustering tendency.* In this task, for a given data set, we assess whether a nonrandom structure exists in the data. Blindly applying a clustering method on a data set will return clusters; however, the clusters mined may be misleading. Clustering analysis on a data set is meaningful only when there is a nonrandom structure in the data.
- *Determining the number of clusters in a data set.* A few algorithms, such as k -means, require the number of clusters in a data set as the parameter. Moreover, the number of clusters can be regarded as an interesting and important summary statistic of a data set. Therefore it is desirable to estimate this number even before a clustering algorithm is used to derive detailed clusters.
- *Measuring clustering quality.* After applying a clustering method on a data set, we want to assess how good the resulting clusters are. A number of measures can be used. Some methods measure how well the clusters fit the data set, while others measure how well the clusters match the ground truth, if such truth is available. There are also measures that score clusterings and thus can compare two sets of clustering results on the same data set.

In this section, we discuss these three topics one by one.

8.5.1 Assessing clustering tendency

Clustering tendency assessment determines whether a given data set has a nonrandom structure, which may lead to meaningful clusters. Consider a data set that does not have any nonrandom structure, such as a set of uniformly distributed points in a data space. Even though a clustering algorithm may return clusters for the data, those clusters are random and thus are not meaningful.

Example 8.8. Clustering requires nonuniform distribution of data. Fig. 8.23 shows a data set that is uniformly distributed in 2-D data space. Although a clustering algorithm may still artificially partition

**FIGURE 8.23**

A data set that is uniformly distributed in the data space.

the points into groups, the groups will unlikely mean anything significant to the application due to the uniform distribution of the data. $\square$

“How can we assess the clustering tendency of a data set?” Intuitively, we can try to measure the probability that the data set is generated by a uniform data distribution. This can be achieved using statistical tests for spatial randomness. To illustrate this idea, let us look at a simple yet effective statistic called the Hopkins statistic.

The **Hopkins Statistic** is a spatial statistic that tests the spatial randomness of a variable as distributed in a space. Given a data set, D , which is regarded as a sample of a random variable, o , we want to determine how far away o is from being uniformly distributed in the data space. We calculate the Hopkins Statistic as follows:

1. Sample n points, $p_1, \dots, p_n$ from the data space. For each point, p_i ($1 \leq i \leq n$), we find the nearest neighbor in D , and let x_i be the distance between p_i and its nearest neighbor in D . That is,

$$x_i = \min_{v \in D} \{dist(p_i, v)\}. \quad (8.26)$$

2. Sample n points, $q_1, \dots, q_n$ uniformly from D without replacement. That is, each point in D has the same probability of being included in this sample, and one point can only be included in the sample at most once. For each q_i ($1 \leq i \leq n$), we find the nearest neighbor of q_i in $D - \{q_i\}$, and let y_i be the distance between q_i and its nearest neighbor in $D - \{q_i\}$. That is,

$$y_i = \min_{v \in D, v \neq q_i} \{dist(q_i, v)\}. \quad (8.27)$$

3. Calculate the Hopkins statistic, H , as

$$H = \frac{\sum_{i=1}^n x_i^d}{\sum_{i=1}^n x_i^d + \sum_{i=1}^n y_i^d}, \quad (8.28)$$

where d is the dimensionality of the data set D .

“What does the Hopkins statistic tell us about how likely data set D follows a uniform distribution in the data space?” If D is uniformly distributed, then $\sum_{i=1}^n y_i^d$ and $\sum_{i=1}^n x_i^d$ are close to each other,

and thus H tends to be about 0.5. However, if D is highly skewed, then the points in D are closer to their nearest neighbors than the random points $p_1, \dots, p_n$ are, and thus $\sum_{i=1}^n x_i^d$ shall be substantially larger than $\sum_{i=1}^n y_i^d$ in expectation, and H tends to be close to 1.

Example 8.9. Hopkins statistic. Consider a 1-D data set $D = \{0.9, 1, 1.3, 1.4, 1.5, 1.8, 2, 2.1, 4.1, 7, 7.4, 7.5, 7.7, 7.8, 7.9, 8.1\}$ in the data space $[0, 10]$. We draw a sample of four points from D without replacement, say, 1.3, 1.8, 7.5, and 7.9. We also draw a sample of four points uniformly from the data space $[0, 10]$, say, 1.9, 4, 6, 8. Then, the Hopkins statistic can be calculated as

$$\begin{aligned} H &= \frac{|1.9 - 2| + |4 - 4.1| + |6 - 7| + |8 - 8.1|}{(|1.9 - 2| + |4 - 4.1| + |6 - 7| + |8 - 8.1|) + (|1.3 - 1.4| + |1.8 - 2| + |7.5 - 7.4| + |7.9 - 7.8|)} \\ &= \frac{1.3}{1.3 + 0.5} = \frac{1.3}{1.8} = 0.72. \end{aligned}$$

Since the Hopkins statistic is substantially larger than 0.5 and is close to 1, the data set D has a strong clustering tendency. Indeed, there are two clusters, one around 1.5 and the other one around 7.8. $\square$

In addition to Hopkins statistic, there are some other methods, such as spatial histogram and distance distribution, comparing statistics between a data set under clustering tendency analysis and the corresponding uniform distribution. For example, distance distribution compares the distribution of pairwise distance in the target data set and that in a random uniform sample from the data space.

8.5.2 Determining the number of clusters

Determining the “right” number of clusters in a data set is important, not only because some clustering algorithms like k -means require such a parameter, but also because the appropriate number of clusters controls the proper granularity of cluster analysis. It can be regarded as finding a good balance between *compressibility* and *accuracy* in cluster analysis. Consider two extreme cases. What if you were to treat the entire data set as a cluster? This would maximize the compression of the data, but such a cluster analysis has no value. In contrast, treating each object in a data set as a cluster gives the finest clustering resolution (i.e., most accurate due to the zero distance between an object and the corresponding cluster center). In some methods like k -means, this even achieves the minimum cost. However, having one object per cluster does not enable any data summarization.

Determining the number of clusters is far from easy, often because the “right” number is ambiguous. Figuring out the right number of clusters often depends on the distribution’s shape and scale in the data set, as well as the clustering resolution required by the user. There are many possible ways to estimate the number of clusters.

For example, a simple method is to set the number of clusters to about $\sqrt{\frac{n}{2}}$ for a data set of n points. In expectation, each cluster has $\sqrt{2n}$ points. Section 8.2.2 introduces the Calinski-Harabasz index, which estimates the number of clusters for k -means.

Let us look at two more alternative methods.

The **elbow method** is based on the observation that increasing the number of clusters can help to reduce the sum of within-cluster variance of each cluster. This is because having more clusters allows one to capture finer groups of data objects that are more similar to each other. However, the marginal

effect of reducing the sum of within-cluster variances may drop if too many clusters are formed, because splitting a cohesive cluster into two gives only a small reduction. Consequently, a heuristic for selecting the right number of clusters is to use the turning point in the curve of the sum of within-cluster variances with respect to the number of clusters.

Technically, given a number, $k > 0$, we can form k clusters on the data set in question using a clustering algorithm like k -means, and calculate the sum of within-cluster variances, $var(k)$. We can then plot the curve of var with respect to k . The first (or most significant) turning point of the curve suggests the “right” number.

More advanced methods can determine the number of clusters using information criteria or information theoretic approaches. Please refer to the bibliographic notes for further information (Section 8.8).

The “right” number of clusters in a data set can also be determined by **cross-validation**, a technique often used in classification (Chapter 6). First, we divide the given data set, D , into m parts. Next, we use $m - 1$ parts to build a clustering model, and use the remaining part to test the quality of the clustering. For example, for each point in the test set, we can find the closest centroid. Consequently, we can use the sum of the squared distances between all points in the test set and the closest centroids to measure how well the clustering model fits the test set. For any integer $k > 0$, we repeat this process m times to derive clusterings of k clusters, using each part in turn as the test set. The average of the quality measure is taken as the overall quality measure. We can then compare the overall quality measure with respect to different values of k and find the number of clusters that best fits the data.

8.5.3 Measuring clustering quality: extrinsic methods

Suppose you have assessed the clustering tendency of a given data set. You may have also tried to predetermine the number of clusters in the set. You can now apply one or multiple clustering methods to obtain clusterings of the data set. *“How good is the clustering generated by a method, and how can we compare the clusterings generated by different methods?”*

Extrinsic vs. intrinsic methods

We have a few methods to choose from for measuring the quality of a clustering. In general, these methods can be categorized into two groups according to whether ground truth is available. Here, *ground truth* is the ideal clustering that is often built using human experts.

If ground truth is available, it can be used by the **extrinsic methods**, which compare the clustering against the ground truth and measure. If the ground truth is unavailable, we can use the **intrinsic methods**, which evaluate the goodness of a clustering by considering how well the clusters are separated. Ground truth can be considered as supervision in the form of “cluster labels.” Hence, extrinsic methods are also known as *supervised methods*, whereas intrinsic methods are *unsupervised methods*.

In this section, we focus on extrinsic methods. We will discuss intrinsic methods in the next section.

Desiderata of extrinsic methods

When the ground truth is available, we can compare it with a clustering to assess the quality of the clustering. Thus the core task in extrinsic methods is to assign a score, $Q(\mathcal{C}, \mathcal{C}_g)$, to a clustering, $\mathcal{C}$, given the ground truth, $\mathcal{C}_g$. Whether an extrinsic method is effective largely depends on the measure, Q , it uses.

In general, a measure Q on clustering quality is effective if it satisfies the following four essential criteria:

- **Cluster homogeneity.** This requires that the purer the clusters in a clustering are, the better the clustering. Suppose that the ground truth says that the objects in a data set, $D = \{a, b, c, d, e, f, g, h\}$, can belong to three categories. Objects a and b are in category L_1 , objects c and d belong to category L_2 , and the others are in category L_3 . Consider clustering, $C_1 = \{\{a, b, c, d\}, \{e, f, g, h\}\}$, wherein a cluster $\{a, b, c, d\} \in C_1$ contains objects from two categories, L_1 and L_2 . Also consider clustering $C_2 = \{\{a, b\}, \{c, d\}, \{e, f, g, h\}\}$, which is identical to C_1 except that C_2 is split into two clusters containing the objects in L_1 and L_2 , respectively. A clustering quality measure, Q , respecting cluster homogeneity should give a higher score to C_2 than C_1 , that is, $Q(C_2, C_g) > Q(C_1, C_g)$.
- **Cluster completeness.** This is the counterpart of cluster homogeneity. Cluster completeness requires that for a clustering, if any two objects belong to the same category according to the ground truth, then they should be assigned to the same cluster. Cluster completeness requires that a clustering should assign objects belonging to the same category (according to the ground truth) to the same cluster. Continue our previous example. Suppose clustering $C_3 = \{\{a, b\}, \{c, d\}, \{e, f\}, \{g, h\}\}$. C_3 and C_2 are identical except that C_3 split the objects in category L_3 into two clusters. Then, a clustering quality measure, Q , respecting cluster completeness should give a higher score to C_2 , that is, $Q(C_2, C_g) > Q(C_1, C_g)$.
- **Rag bag.** In many practical scenarios, there is often a “rag bag” category containing objects that cannot be merged with other objects. Such a category is often called “miscellaneous,” “other,” and so on. The rag bag criterion states that putting a heterogeneous object into a pure cluster should be penalized more than putting it into a rag bag. Consider a clustering C_1 and a cluster $C \in C_1$ such that all objects in C except for one, denoted by o , belong to the same category according to the ground truth. Consider a clustering C_2 identical to C_1 except that o is assigned to a cluster $C' \neq C$ in C_2 such that C' contains objects from various categories according to ground truth, and thus is noisy. In other words, C' in C_2 is a rag bag. Then, a clustering quality measure Q respecting the rag bag criterion should give a higher score to C_2 , that is, $Q(C_2, C_g) > Q(C_1, C_g)$.
- **Small cluster preservation.** If a small category is split into small pieces in a clustering, those small pieces may likely become noise and thus the small category cannot be discovered from the clustering. The small cluster preservation criterion states that splitting a small category into pieces is more harmful than splitting a large category into pieces. Consider an extreme case. Let D be a data set of $n + 2$ objects such that, according to ground truth, n objects, denoted by $o_1, \dots, o_n$, belong to one category and the other two objects, denoted by o_{n+1}, o_{n+2} , belong to another category. Suppose clustering C_1 has three clusters, $C_1^1 = \{o_1, \dots, o_n\}$, $C_1^2 = \{o_{n+1}\}$, and $C_1^3 = \{o_{n+2}\}$. Let clustering C_2 have three clusters, too, namely $C_2^1 = \{o_1, \dots, o_{n-1}\}$, $C_2^2 = \{o_n\}$, and $C_2^3 = \{o_{n+1}, o_{n+2}\}$. In other words, C_1 splits the small category and C_2 splits the big category. A clustering quality measure Q preserving small clusters should give a higher score to C_2 , that is, $Q(C_2, C_g) > Q(C_1, C_g)$.

Categories of extrinsic methods

The ground truth may be used in different ways to evaluate clustering quality, which lead to different extrinsic methods. In general, the extrinsic methods can be categorized according to how the ground truth is used as follows.

- **The matching-based methods** examine how well the clustering results match the ground truth in partitioning the objects in the data set. For example, the purity methods assess how a cluster matches only those objects in one group in the ground truth.
- **The information theory-based methods** compare the distribution of the clustering results and that of the ground truth. Entropy or other measures in information theory are often employed to quantify the comparison. For example, we can measure the conditional entropy between the clustering results and the ground truth to measure whether there exists dependency between the information of the clustering results and the ground truth. The higher the dependency, the better the clustering results.
- **The pairwise comparison-based methods** treat each group in the ground truth as a class and then check the pairwise consistency of the objects in the clustering results. The clustering results are good if more pairs of objects of the same class are put into the same cluster, less pairs of objects of different classes are put into the same cluster, and less pairs of objects of the same class are put into different clusters.

Next, let us use some examples to illustrate the above categories of extrinsic methods.

Matching-based methods

The matching-based methods compare clusters in the clustering results and the groups in the ground truth. Let us use an example to explain the ideas.

Suppose a clustering method partitions a set of objects $D = \{o_1, \dots, o_n\}$ into clusters $\mathcal{C} = \{C_1, \dots, C_m\}$. The ground truth $\mathcal{G}$ also partitions the same set of objects into groups $\mathcal{G} = \{G_1, \dots, G_l\}$. Let $C(o_x)$ and $G(o_x)$ ($1 \leq x \leq n$) be the cluster-id and the group-id of object o_x in the clustering results and the ground truth, respectively.

For a cluster C_i ($1 \leq i \leq m$), how well C_i matches group G_j in the ground truth can be measured by $|C_i \cap G_j|$, the larger the better. $\frac{|C_i \cap G_j|}{|C_i|}$ can be regarded as the purity of cluster C_i , where G_j matching C_i maximizes $|C_i \cap G_j|$. The purity of the whole clustering results can be calculated as the weighted sum of the purity of the clusters. That is,

$$purity = \sum_{i=1}^m \frac{|C_i|}{n} \max_{j=1}^l \left\{ \frac{|C_i \cap G_j|}{|C_i|} \right\} = \frac{1}{n} \sum_{i=1}^m \max_{j=1}^l \{|C_i \cap G_j|\}. \quad (8.29)$$

The higher the purity, the purer are the clusters, that is, the more objects in each cluster belong to the same group in the ground truth. When the purity is 1, each cluster either matches a group perfectly or is a subset of a group. In other words, no two objects belong to two groups are mixed in one cluster. However, it is possible that multiple clusters partition a group in the ground truth.

Example 8.10. Purity. Consider the set of objects $D = \{a, b, c, d, e, f, g, h, i, j, k\}$. The clustering ground truth and two clusterings $\mathcal{C}_1$ and $\mathcal{C}_2$ output by two methods are shown in Table 8.1.

The purity of clustering $\mathcal{C}_1$ is calculated by $\frac{1}{11} \times (4 + 2 + 4 + 1) = \frac{11}{11} = 1$ and that of clustering $\mathcal{C}_2$ is $\frac{1}{11} (2 + 3 + 1) = \frac{6}{11}$. In terms of purity, $\mathcal{C}_1$ is better than $\mathcal{C}_2$. Please note that, although $\mathcal{C}_1$ has purity 1, it splits G_1 in the ground truth into two clusters, C_1 and C_2 . $\square$

There are some other matching based methods further refine the measurement of matching quality, such as maximum matching and using F-measure.

Table 8.1 A set of objects, the clustering ground truth, and two clusterings.

Object	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>j</i>	<i>k</i>
Ground truth $\mathcal{G}$	G_1	G_1	G_1	G_1	G_1	G_1	G_2	G_2	G_2	G_2	G_3
Clustering $\mathcal{C}_1$	C_1	C_1	C_1	C_1	C_2	C_2	C_3	C_3	C_3	C_3	C_4
Clustering $\mathcal{C}_2$	C_1	C_1	C_2	C_2	C_2	C_3	C_1	C_2	C_2	C_1	C_3

Information theory–based methods

A clustering assigns objects to clusters and thus can be regarded as a compression of the information carried by the objects. In other words, a clustering can be regarded as a compressed representation of a given set of objects. Therefore we can use information theory to compare a clustering and the ground truth as representations. This is the general idea behind the information theory–based methods.

For example, we can measure the amount of information needed to describe the ground truth given the distribution of a clustering output by a method. Better the clustering results approach the ground truth, less amount information is needed. This leads to a natural approach using conditional entropy.

Concretely, according to information theory, the entropy of a clustering $\mathcal{C}$ is

$$H(\mathcal{C}) = - \sum_{i=1}^m \frac{|C_i|}{n} \log \frac{|C_i|}{n},$$

and the entropy of the ground truth is

$$H(\mathcal{G}) = - \sum_{i=1}^l \frac{|G_i|}{n} \log \frac{|G_i|}{n}.$$

The conditional entropy of $\mathcal{G}$ given cluster C_i is

$$H(\mathcal{G}|C_i) = - \sum_{j=1}^l \frac{|C_i \cap G_j|}{|C_i|} \log \frac{|C_i \cap G_j|}{|C_i|}.$$

The conditional entropy of $\mathcal{G}$ given clustering $\mathcal{C}$ is

$$H(\mathcal{G}|\mathcal{C}) = \sum_{i=1}^m \frac{|C_i|}{n} H(\mathcal{G}|C_i) = - \sum_{i=1}^m \sum_{j=1}^l \frac{|C_i \cap G_j|}{n} \log \frac{|C_i \cap G_j|}{|C_i|}.$$

In addition to the simple conditional entropy, more sophisticated information theory–based measures may be used, such as normalized mutual information and variation of information.

Taking the case in Table 8.1 as an example, we can calculate

$$H(\mathcal{G}|\mathcal{C}_1) = - \left(\frac{4}{11} \log \frac{4}{4} + \frac{2}{11} \log \frac{2}{2} + \frac{4}{11} \log \frac{4}{4} + \frac{1}{11} \log \frac{1}{1} \right) = 0$$

and

$$H(\mathcal{G}|\mathcal{C}_2) = -\left(\frac{2}{11} \log \frac{2}{4} + \frac{2}{11} \log \frac{2}{4} + \frac{3}{11} \log \frac{3}{5} + \frac{2}{11} \log \frac{2}{5} + \frac{1}{11} \log \frac{1}{2} + \frac{1}{11} \log \frac{1}{2}\right) \\ = 0.297.$$

Clustering $\mathcal{C}_1$ has better quality than $\mathcal{C}_2$ in terms of conditional entropy. Again, although $H(\mathcal{G}|\mathcal{C}_1) = 0$, conditional entropy cannot detect the issue that $\mathcal{C}_1$ splits the objects in G_1 into two clusters.

Pairwise comparison-based methods

The pairwise comparison-based methods treat each group in the ground truth as a class. For each pair of objects $o_i, o_j \in D$ ($1 \leq i, j \leq n, i \neq j$), if they are assigned to the same cluster/group, the assignment is regarded as positive, and otherwise, negative. Then, depending on assignments of o_i and o_j into clusters $C(o_i)$, $C(o_j)$, $G(o_i)$, and $G(o_j)$, we have four possible cases.

	$C(o_i) = C(o_j)$	$C(o_i) \neq C(o_j)$
$G(o_i) = G(o_j)$	true positive	false negative
$G(o_i) \neq G(o_j)$	false positive	true negative

Using the statistics on pairwise comparison, we can assess the quality of the clustering results approaching the ground truth. For example, we can use the Jaccard coefficient, which is defined as

$$J = \frac{\text{true positive}}{\text{true positive} + \text{false negative} + \text{false positive}}.$$

Many other measures can be built based on the pairwise comparison statistics, such as Rand statistic, fowlkes-Mallows measure, BCubed precision, and recall. The pairwise comparison results can be further used to conduct correlation analysis. For example, we can form a binary matrix $\mathbf{G}$ according to the ground truth, where element $v_{ij} = 1$ if $G(o_i) = G(o_j)$, and otherwise 0. A binary matrix $\mathbf{C}$ can also be constructed in a similar way based on a clustering $\mathcal{C}$. We can analyze the element-wise correlation between the two matrixes and use the correlation to measure the quality of the clustering results. Clearly, the more correlated the two matrixes, the better the clustering results.

8.5.4 Intrinsic methods

When the ground truth of a data set is not available, we have to use an intrinsic method to assess the clustering quality. Unable to reference any external supervision information, the intrinsic methods have to come back to the fundamental intuition in clustering analysis, that is, examining how compact clusters are and how well clusters are separated. Many intrinsic methods take the advantage of a similarity or distance measure between objects in the data set.

For example, the **Dunn index** measures the compactness of clusters by the maximum distance between two points that belong to the same cluster, that is, $\Delta = \max_{C(o_i)=C(o_j)}\{d(o_i, o_j)\}$. It measures the degree of separation among different clusters by the minimum distance between two points that belong to different clusters, that is $\delta = \min_{C(o_i) \neq C(o_j)}\{d(o_i, o_j)\}$. Then, the Dunn index is simply the ration $DI = \frac{\delta}{\Delta}$. The larger the ratio, the farther away the clusters are separated comparing to the compactness of the clusters.

The Dunn index uses the extreme distances to measure the cluster compactness and intercluster separation. The measures δ and Δ may be affected by the outliers. Many methods consider the average situations. The **silhouette coefficient** is such a measure. For a data set, D , of n objects, suppose D is partitioned into k clusters, $C_1, \dots, C_k$. For each object $\mathbf{o} \in D$, we calculate $a(\mathbf{o})$ as the average distance between $\mathbf{o}$ and all other objects in the cluster to which $\mathbf{o}$ belongs. Similarly, $b(\mathbf{o})$ is the minimum average distance from $\mathbf{o}$ to all clusters to which $\mathbf{o}$ does not belong. Formally, suppose $\mathbf{o} \in C_i$ ($1 \leq i \leq k$). Then,

$$a(\mathbf{o}) = \frac{\sum_{\mathbf{o}' \in C_i, \mathbf{o}' \neq \mathbf{o}} \text{dist}(\mathbf{o}, \mathbf{o}')}{|C_i| - 1} \quad (8.30)$$

and

$$b(\mathbf{o}) = \min_{C_j: 1 \leq j \leq k, j \neq i} \left\{ \frac{\sum_{\mathbf{o}' \in C_j} \text{dist}(\mathbf{o}, \mathbf{o}')}{|C_j|} \right\}. \quad (8.31)$$

The **silhouette coefficient** of $\mathbf{o}$ is then defined as

$$s(\mathbf{o}) = \frac{b(\mathbf{o}) - a(\mathbf{o})}{\max\{a(\mathbf{o}), b(\mathbf{o})\}}. \quad (8.32)$$

The value of the silhouette coefficient is between -1 and 1 . The value of $a(\mathbf{o})$ reflects the compactness of the cluster to which $\mathbf{o}$ belongs. The smaller the value, the more compact the cluster. The value of $b(\mathbf{o})$ captures the degree to which $\mathbf{o}$ is separated from other clusters. The larger $b(\mathbf{o})$ is, the more separated $\mathbf{o}$ is from other clusters. Therefore when the silhouette coefficient value of $\mathbf{o}$ approaches 1 , the cluster containing $\mathbf{o}$ is compact and $\mathbf{o}$ is far away from other clusters, which is the preferable case. However, when the silhouette coefficient value is negative (i.e., $b(\mathbf{o}) < a(\mathbf{o})$), this means that, in expectation, $\mathbf{o}$ is closer to the objects in another cluster than to the objects in the same cluster as $\mathbf{o}$. In many cases, this is a bad situation and should be avoided.

To measure a cluster's fitness within a clustering, we can compute the average silhouette coefficient value of all objects in the cluster. To measure the quality of a clustering, we can use the average silhouette coefficient value of all objects in the data set. The silhouette coefficient and other intrinsic measures can also be used in the elbow method to heuristically derive the number of clusters in a data set by replacing the sum of within-cluster variances.

8.6 Summary

- A **cluster** is a collection of data objects that are *similar* to one another within the same cluster and are *dissimilar* to the objects in other clusters. The process of grouping a set of physical or abstract objects into classes of *similar* objects is called **clustering**.
- Cluster analysis has extensive **applications**, including business intelligence, image pattern recognition, Web search, biology, and security. Cluster analysis can be used as a standalone data mining tool to gain insight into the data distribution or as a preprocessing step for other data mining algorithms operating on the detected clusters.
- Clustering is a dynamic field of research in data mining. It is related to **unsupervised learning** in machine learning.

- Clustering is a challenging field. Typical **requirements** of it include scalability, the ability to deal with different types of data and attributes, the discovery of clusters in arbitrary shape, minimal requirements for domain knowledge to determine input parameters, the ability to deal with noisy data, incremental clustering and insensitivity to input order, the capability of clustering high-dimensionality data, constraint-based clustering, and interpretability and usability.
- Many clustering algorithms have been developed. These can be categorized from several **orthogonal aspects** such as those regarding partitioning criteria, separation of clusters, similarity measures used, and clustering space. This chapter discusses major fundamental clustering methods of the following categories: *partitioning methods*, *hierarchical methods*, and *density-based and grid-based methods*. Some algorithms may belong to more than one category.
- A **partitioning method** first creates an initial set of k partitions, where parameter k is the number of partitions to construct. It then uses an *iterative relocation technique* that attempts to improve the partitioning by moving objects from one group to another. Typical partitioning methods include k -means, k -medoids, and k -modes.
- The **centroid-based partitioning technique** uses the **within-cluster variation** to measure the quality of clusters, which is the sum of squared error between all objects in a cluster and the centroid of the cluster. Minimizing the within-cluster variation is computationally challenging and thus some greedy approaches are often used. The **k -means method** uses the mean value of the points within a cluster as the centroid. It randomly selects k objects as the initial centroids of the clusters and then iteratively conducts object assignment and mean update steps until the assignment becomes stable or a certain number of iterations are reached.
- As a variation of k -means to overcome the effect of outliers, the **k -medoids method** uses actual objects to represent clusters. While k -medoids is more robust against noise and outliers, it incurs higher computational cost in each iteration. As another variation of k -means, the **k -modes method** uses modes to measure the similarity on nominal data. We can also use kernel functions, such as the Gaussian radial basis function, in k -means to find clusters that are concave and not linearly separable.
- A **hierarchical method** creates a hierarchical decomposition of the given set of data objects. The method can be classified as being either *agglomerative (bottom-up)* or *divisive (top-down)*, based on how the hierarchical decomposition is formed. **Linkage measures** can be used to assess the distance between clusters in hierarchical clustering. Some widely used measures include *minimum distance (single-linkage)*, *maximum distance (complete-linkage)*, *mean distance*, and *average distance*. The **Lance-Williams algorithm** generalizes different measures and the agglomerative hierarchical clustering framework. The **minimum spanning tree based approach** is a representative method for divisive hierarchical clustering. Hierarchical clustering results can be represented using a **dendrogram**.
- **BIRCH** is a method combining hierarchical clustering and other clustering methods. In BIRCH, clusters are represented using **clustering features** (CF for short) and a hierarchical clustering is represented by a **CF-tree**.
- To overcome some of the drawbacks of hierarchical clustering methods, **probabilistic hierarchical clustering** uses probabilistic models to measure distances between clusters. It shares the same framework and thus has the same efficiency as agglomerative hierarchical clustering methods.
- A **density-based method** clusters objects based on the notion of density. It grows clusters either according to the density of neighborhood objects (e.g., in DBSCAN) or according to a density

function (e.g., in DENCLUE). OPTICS is a density-based method that generates an augmented ordering of the data's clustering structure.

- A **grid-based method** first quantizes the object space into a finite number of cells that form a grid structure, and then performs clustering on the grid structure. STING is a typical example of a grid-based method based on statistical information stored in grid cells. CLIQUE is a grid-based and subspace clustering algorithm.
- **Clustering evaluation** assesses the feasibility of clustering analysis on a data set and the quality of the results generated by a clustering method. The major tasks include assessing clustering tendency, determining the number of clusters, and measuring clustering quality.
- Some statistics, such as **Hopkins statistic**, can be used to assess clustering tendency. In addition to the Calinski-Harabasz index, the **elbow method**, and the **cross-validation** technique can be used to decide the number of clusters in a data set. Depending on whether the ground truth is available, the methods measuring clustering quality can be divided into **extrinsic methods** and **intrinsic methods**. The extrinsic methods try to address the desiderata of *cluster homogeneity*, *cluster completeness*, *rag bag*, and *small cluster preservation*. The extrinsic methods can be divided into the **matching-based methods** (e.g., *purity*), the **information theory-based methods** (e.g., *entropy*), and the **pairwise comparison-based methods** (e.g., using Jaccard coefficient). Examples of the intrinsic methods are the **Dunn index** and the **silhouette coefficient**.

8.7 Exercises

- 8.1. Briefly describe and give examples of each of the following approaches to clustering: *partitioning* methods, *hierarchical* methods, *density-based* and *grid-based* methods, and *bi-clustering* methods.
- 8.2. Suppose that the data mining task is to cluster points (with (x, y) representing location) into three clusters, where the points are

$$A_1(2, 10), A_2(2, 5), A_3(8, 4), B_1(5, 8), B_2(7, 5), B_3(6, 4), C_1(1, 2), C_2(4, 9).$$

The distance function is Euclidean distance. Suppose initially we assign A_1 , B_1 , and C_1 as the center of each cluster, respectively. Use the *k-means* algorithm to show *only*

- a. The three cluster centers after the first round of execution.
 - b. The final three clusters.
- 8.3. Use an example to show why the *k-means* algorithm may not find the global optimum, that is, optimizing the within-cluster variation.
 - 8.4. For the *k-means* algorithm, it is interesting to note that by choosing the initial cluster centers carefully, we may be able to not only speed up the algorithm's convergence, but also guarantee the quality of the final clustering. The **k-means++** algorithm is a variant of *k-means*, which chooses the initial centers as follows. First, it selects one center uniformly at random from the objects in the data set. Iteratively, for each object p other than the chosen center, it chooses an object as the new center. This object is chosen at random with probability proportional to $\text{dist}(p)^2$, where $\text{dist}(p)$ is the distance from p to the closest center that has already been chosen. The iteration continues until k centers are selected.

Explain why this method will not only speed up the convergence of the k -means algorithm, but also guarantee the quality of the final clustering results.

- 8.5. Provide the pseudocode of the object reassignment step of the PAM algorithm.
- 8.6. Both k -means and k -medoids algorithms can perform effective clustering.
 - a. Illustrate the strength and weakness of k -means in comparison with k -medoids.
 - b. Illustrate the strength and weakness of these schemes in comparison with a hierarchical clustering scheme.
- 8.7. Show that the single-linkage method is equivalent to taking $\alpha_i = \alpha_j = 0.5$, $\beta = 0$, and $\gamma = -0.5$ in the Lance-Williams formula; the complete-linkage method is equivalent to $\alpha_i = \alpha_j = 0.5$, $\beta = 0$, and $\gamma = 0.5$; and the Ward's criterion is equivalent to $\alpha_i = \frac{n_i + n_k}{n_i + n_j + n_k}$, $\alpha_j = \frac{n_j + n_k}{n_i + n_j + n_k}$, $\beta = -\frac{n_k}{n_i + n_j + n_k}$, and $\gamma = 0$.
- 8.8. Prove that in DBSCAN*, the density-connectedness is an equivalence relation.
- 8.9. Prove that in DBSCAN*, for a fixed *MinPts* value and two neighborhood thresholds, $\epsilon_1 < \epsilon_2$, a cluster C with respect to ϵ_1 and *MinPts* must be a subset of a cluster C' with respect to ϵ_2 and *MinPts*.
- 8.10. Provide the pseudocode of the OPTICS algorithm.
- 8.11. Why is it that BIRCH encounters difficulties in finding clusters of arbitrary shape but OPTICS does not? Propose modifications to BIRCH to help it find clusters of arbitrary shape.
- 8.12. Provide the pseudocode of the step in CLIQUE that finds dense cells in all subspaces.
- 8.13. Present conditions under which density-based clustering is more suitable than partitioning-based clustering and hierarchical clustering. Give application examples to support your argument.
- 8.14. Give an example of how specific clustering methods can be *integrated*, for example, where one clustering algorithm is used as a preprocessing step for another. In addition, provide reasoning as to why the integration of two methods may sometimes lead to improved clustering quality and efficiency.
- 8.15. Clustering is recognized as an important data mining task with broad applications. Give one application example for each of the following cases:
 - a. An application that uses clustering as a major data mining function.
 - b. An application that uses clustering as a preprocessing tool for data preparation for other data mining tasks.
- 8.16. Data cubes and multidimensional databases contain nominal, ordinal, and numeric data in hierarchical or aggregate forms. Based on what you have learned about the clustering methods, design a clustering method that finds clusters in large data cubes effectively and efficiently.
- 8.17. Describe each of the following clustering algorithms in terms of the following criteria: (1) shapes of clusters that can be determined; (2) input parameters that must be specified; and (3) limitations.
 - a. k -means
 - b. k -medoids
 - c. BIRCH
 - d. DBSCAN*
- 8.18. Human eyes are fast and effective at judging the quality of clustering methods for 2-D data. Can you design a data visualization method that may help humans visualize data clusters and judge the clustering quality for 3-D data? What about for even higher-dimensional data?

- 8.19.** Discuss how well purity, entropy, and the method using Jaccard coefficient satisfy the four essential requirements for extrinsic clustering evaluation methods.

8.8 Bibliographic notes

Clustering has been extensively studied for over 40 years and across many disciplines due to its broad applications. Most books on pattern classification and machine learning contain chapters on cluster analysis or unsupervised learning. Several textbooks are dedicated to the methods of cluster analysis, including Hartigan [Har75]; Jain and Dubes [JD88]; Kaufman and Rousseeuw [KR90]; and Arabie, Hubert, and De Sorte [AHS96]. There are also many survey articles on different aspects of clustering methods. Recent ones include Jain, Murty, and Flynn [JMF99]; Parsons, Haque, and Liu [PHL04]; Xu and Wunsch [XW05]; Jain [Jai10]; Greenlaw and Kantabutra [GK13]; Xu and Tian [XT15]; and Berkhin [Ber06].

For partitioning methods, the k -means algorithm was first introduced by Lloyd [Llo57], and then by MacQueen [Mac67]. Arthur and Vassilvitskii [AV07] presented the k -means++ algorithm. A filtering algorithm, which uses a spatial hierarchical data index to speed up the computation of cluster means, is given in Kanungo et al. [KMN⁺02].

The k -medoids algorithms of PAM and CLARA were proposed by Kaufman and Rousseeuw [KR90]. The k -modes (for clustering nominal data) and k -prototypes (for clustering hybrid data) algorithms were proposed by Huang [Hua98]. The k -modes clustering algorithm was also proposed independently by Chaturvedi, Green, and Carroll [CGC94, CGC01]. The CLARANS algorithm was proposed by Ng and Han [NH94]. Ester, Kriegel, and Xu [EKX95] proposed techniques for further improvement of the performance of CLARANS using efficient spatial access methods such as R^* -tree and focusing techniques. A k -means-based scalable clustering algorithm was proposed by Bradley, Fayyad, and Reina [BFR98]. The kernel k -means method was developed by Dhillon, Guan, and Kulis [DGK04].

An early survey of agglomerative hierarchical clustering algorithms was conducted by Day and Edelsbrunner [DE84]. Murtagh and Contreras [MC12] provided a more recent survey on hierarchical clustering. Zhao, Karypis, and Fayyad [ZKF05] surveyed the hierarchical clustering algorithms for document databases. Agglomerative hierarchical clustering, such as AGNES, and divisive hierarchical clustering, such as DIANA, were introduced by Kaufman and Rousseeuw [KR90]. Rohlf [Roh73] developed the essential idea of hierarchical clustering using the minimum spanning tree. An interesting direction for improving the clustering quality of hierarchical clustering methods is to integrate hierarchical clustering with distance-based iterative relocation or other nonhierarchical clustering methods. For example, BIRCH, by Zhang, Ramakrishnan, and Livny [ZRL96], first performs hierarchical clustering with a CF-tree before applying other techniques. Hierarchical clustering can also be performed by sophisticated linkage analysis, transformation, or nearest-neighbor analysis, such as CURE by Guha, Rastogi, and Shim [GRS98]; ROCK (for clustering nominal attributes) by Guha, Rastogi, and Shim [GRS99]; and Chameleon by Karypis, Han, and Kumar [KHK99].

Ward [War63] proposed the Ward's criterion. Murtagh and Legendre [ML14] surveyed how the Ward's criterion is implemented. Lance and Williams [LW67] proposed the Lance-Williams algorithm. A probabilistic hierarchical clustering framework following normal linkage algorithms and using probabilistic models to define cluster similarity was developed by Friedman [Fri03] and Heller and Ghahramani [HG05].

For density-based clustering methods, DBSCAN was proposed by Ester, Kriegel, Sander, and Xu [EKSX96]. Campello, Moulavi, Zimek, and Sander [CMZS15] developed both DBSCAN* and HDBSCAN. Ankerst, Breunig, Kriegel, and Sander [ABKS99] developed OPTICS, a cluster-ordering method that facilitates density-based clustering without worrying about parameter specification. The DENCLUE algorithm, based on a set of density distribution functions, was proposed by Hinneburg and Keim [HK98]. Hinneburg and Gabriel [HG07] developed DENCLUE 2.0, which includes a new hill-climbing procedure for Gaussian kernels that adjusts the step size automatically.

STING, a grid-based multiresolution approach that collects statistical information in grid cells, was proposed by Wang, Yang, and Muntz [WYM97]. WaveCluster, developed by Sheikholeslami, Chatterjee, and Zhang [SCZ98], is a multiresolution clustering approach that transforms the original feature space by wavelet transform.

Scalable methods for clustering nominal data were studied by Gibson, Kleinberg, and Raghavan [GKR98]; Guha, Rastogi, and Shim [GRS99]; and Ganti, Gehrke, and Ramakrishnan [GGR99]. There are also many other clustering paradigms. For example, fuzzy clustering methods are discussed in Kaufman and Rousseeuw [KR90], Bezdek [Bez81], and Bezdek and Pal [BP92].

For high-dimensional clustering, an Apriori-based dimension-growth subspace clustering algorithm called CLIQUE was proposed by Agrawal, Gehrke, Gunopulos, and Raghavan [AGGR98]. It integrates density-based and grid-based clustering methods.

Recent studies have proceeded to clustering stream data (Babcock et al. [BBD⁺02]). A k -median-based data stream clustering algorithm was proposed by Guha, Mishra, Motwani, and O'Callaghan [GMMO00] and by O'Callaghan et al. [OMM⁺02]. A method for clustering evolving data streams was proposed by Aggarwal, Han, Wang, and Yu [AHWY03]. A framework for projected clustering of high-dimensional data streams was proposed by Aggarwal, Han, Wang, and Yu [AHWY04].

Clustering evaluation is discussed in a few monographs and survey articles such as Jain and Dubes [JD88] and Halkidi, Batistakis, and Vazirgiannis [HBV01]. The Hopkins statistic was proposed by Hopkins and Skellam [HS54]. The problem of determining the number of clusters in a data set was discussed by Sugar and James [SJ03] and Cordeiro De Amorim and Hennig [CH15b], for example.

The extrinsic methods for clustering quality evaluation are extensively explored. Some recent studies include Meilă [Mei03, Mei05] and Amigó, Gonzalo, Artiles, and Verdejo [AGAV09]. The four essential criteria introduced in this chapter are formulated in Amigó, Gonzalo, Artiles, and Verdejo [AGAV09], whereas some individual criteria were also mentioned earlier, for example, in Meilă [Mei03] and Rosenberg and Hirschberg [RH07]. Bagga and Baldwin [BB98] introduced the BCubed metrics. The silhouette coefficient is described in Kaufman and Rousseeuw [KR90].